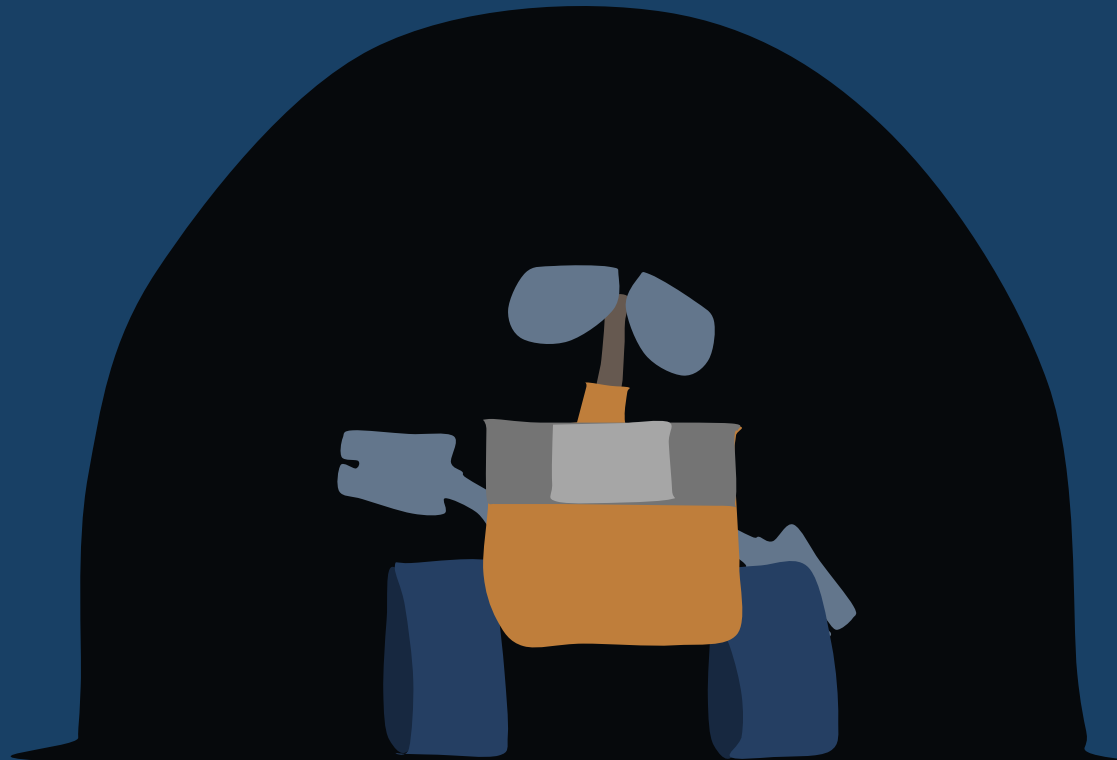


Real-Time 3D Reconstruction and 6-DoF Tracking with an Event Camera

Hanme Kim, Stefan Leutenegger, and Andrew J. Davison

Context & Relevance

- Suppose you have a robot using SLAM to navigate a tunnel.



Context & Relevance

- Suppose you have a robot using SLAM to navigate a tunnel.
- To find out its location, it takes a picture of its current view and tries extract features from it.



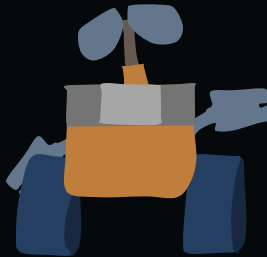
Context & Relevance

- Suppose you have a robot using SLAM to navigate a tunnel.
- To find out its location, it takes a picture of its current view and tries extract features from it.



Context & Relevance

- However, as it reaches the end of the tunnel...



Context & Relevance

- However, as it reaches the end of the tunnel...



Context & Relevance

- However, as it reaches the end of the tunnel, this is the input it receives about its environment



Context & Relevance

- Its sensor's low dynamic range prevents it from capturing edges and texture from the frame.



Context & Relevance

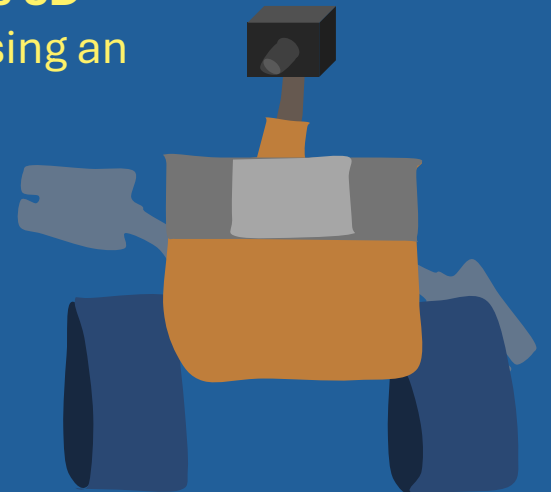
- How can we address issues like these that commonly happen as a result of using standard cameras as input?

Context & Relevance

- How can we address issues like these that commonly happen as a result of using standard cameras as input?

Event cameras!

The authors propose using an **event camera** in the context of SLAM (namely, the authors claim that they can create a **real-time 3D reconstruction** of a scene and also **track camera motion** using an event camera).



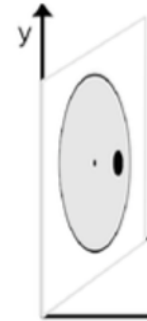
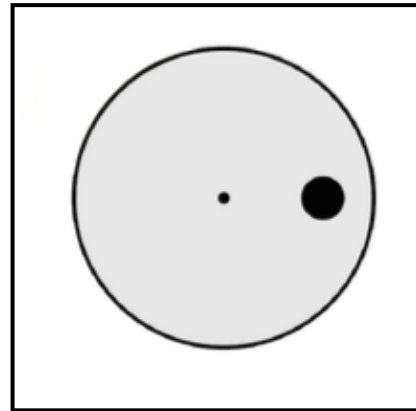
Structure of this presentation

- Refresher on event cameras
- Exploring event cameras with SLAM
- Explanation of the method:
 - Three probabilistic filters:
 - Filter 1: predicts camera pose
 - Filter 2: predicts per-pixel gradient + intensity reconstruction
 - Filter 3: predicts inverse-depth
- Description of the experiments
- Summary of discussion, conclusion, and key takeaways

Event cameras

- Conventional cameras take a complete snapshot of every pixel at once (i.e., a frame) every few milliseconds

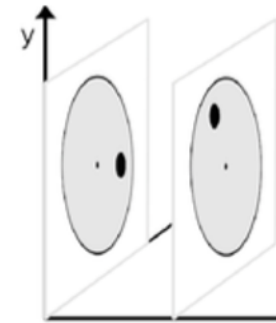
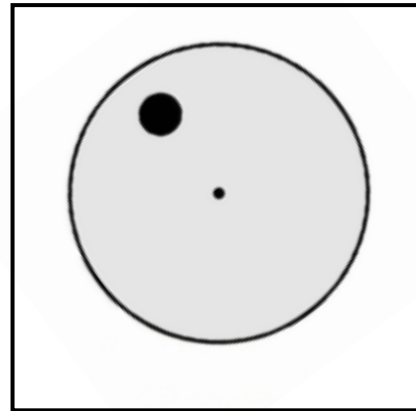
Intensity at each
pixel logged into
the snapshot



Event cameras

- Conventional cameras take a complete snapshot of every pixel at once (i.e., a frame) every few milliseconds

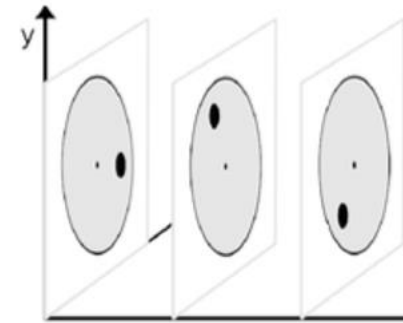
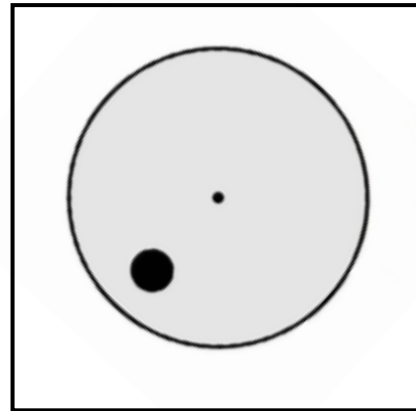
Intensity at each
pixel logged into
the snapshot



Event cameras

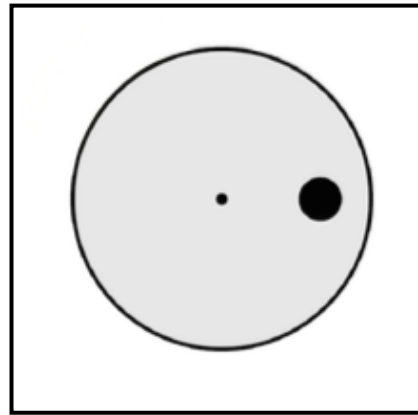
- Conventional cameras take a complete snapshot of every pixel at once (i.e., a frame) every few milliseconds

Intensity at each
pixel logged into
the snapshot



Event cameras

- In contrast, event cameras record changes in pixel intensity
- More specifically, pixels in event cameras asynchronously report log intensity changes of a pre-set threshold size as a stream of asynchronous events
 - Each change in pixel intensity is called an **event**



Standard cameras



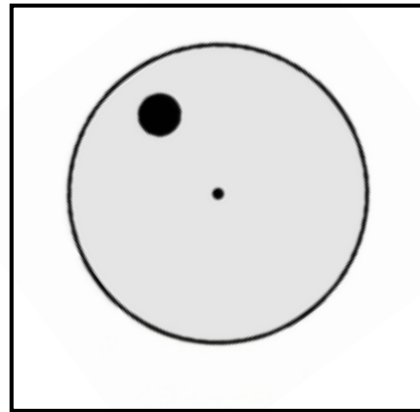
Event cameras

Event cameras

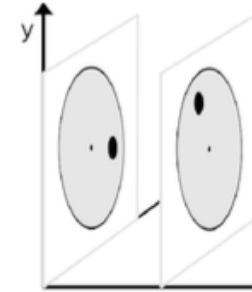
- In contrast, event cameras record changes in pixel intensity
- More specifically, pixels in event cameras asynchronously report log intensity changes of a pre-set threshold size as a stream of asynchronous events
 - Each change in pixel intensity is called an **event**

A negative event: negative gradient
(i.e., pixel goes from a lighter intensity to a darker intensity)

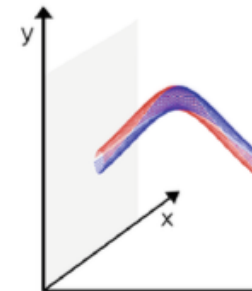
A positive event: positive gradient
(i.e., pixel goes from a darker intensity to a lighter intensity)



Standard cameras



Event cameras

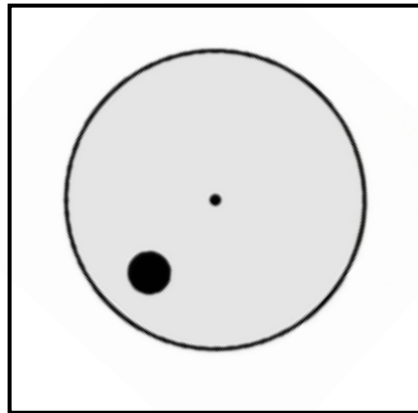


Event cameras

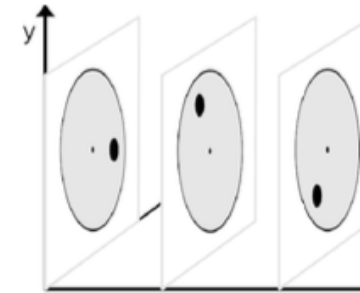
- In contrast, event cameras record changes in pixel intensity
- More specifically, pixels in event cameras asynchronously report log intensity changes of a pre-set threshold size as a stream of asynchronous events
 - Each change in pixel intensity is called an **event**

A negative event: negative gradient
(i.e., pixel goes from a lighter intensity to a darker intensity)

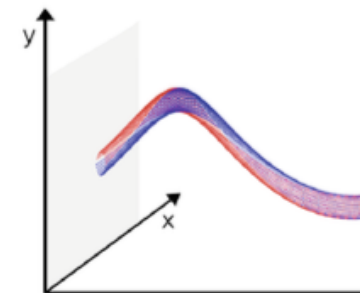
A positive event: positive gradient
(i.e., pixel goes from a darker intensity to a lighter intensity)



Standard cameras

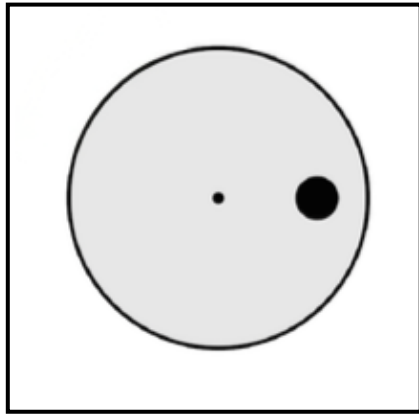


Event cameras

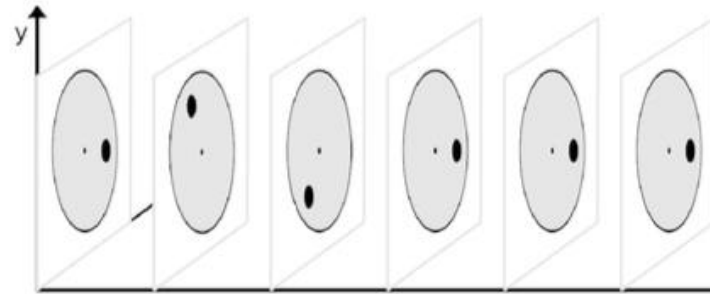


Event cameras

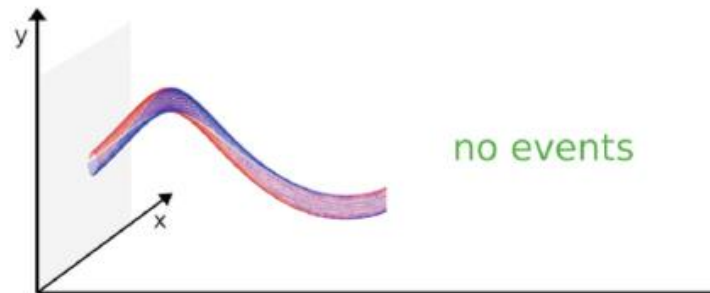
- When there are no changes in intensities (i.e., a static pixel), then there are no events to track



Standard cameras



Event cameras



Event cameras

- But when there is rapid motion, we get high frequencies of event firing

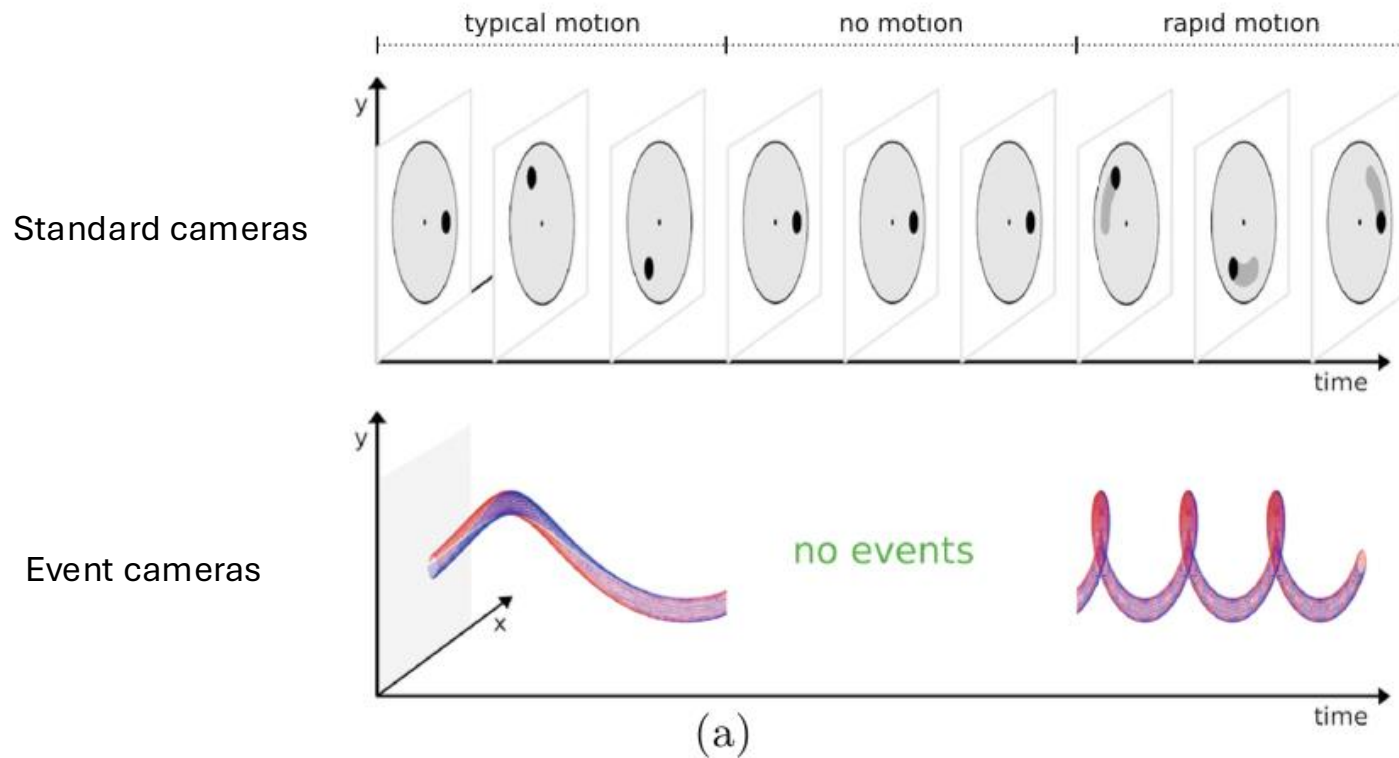
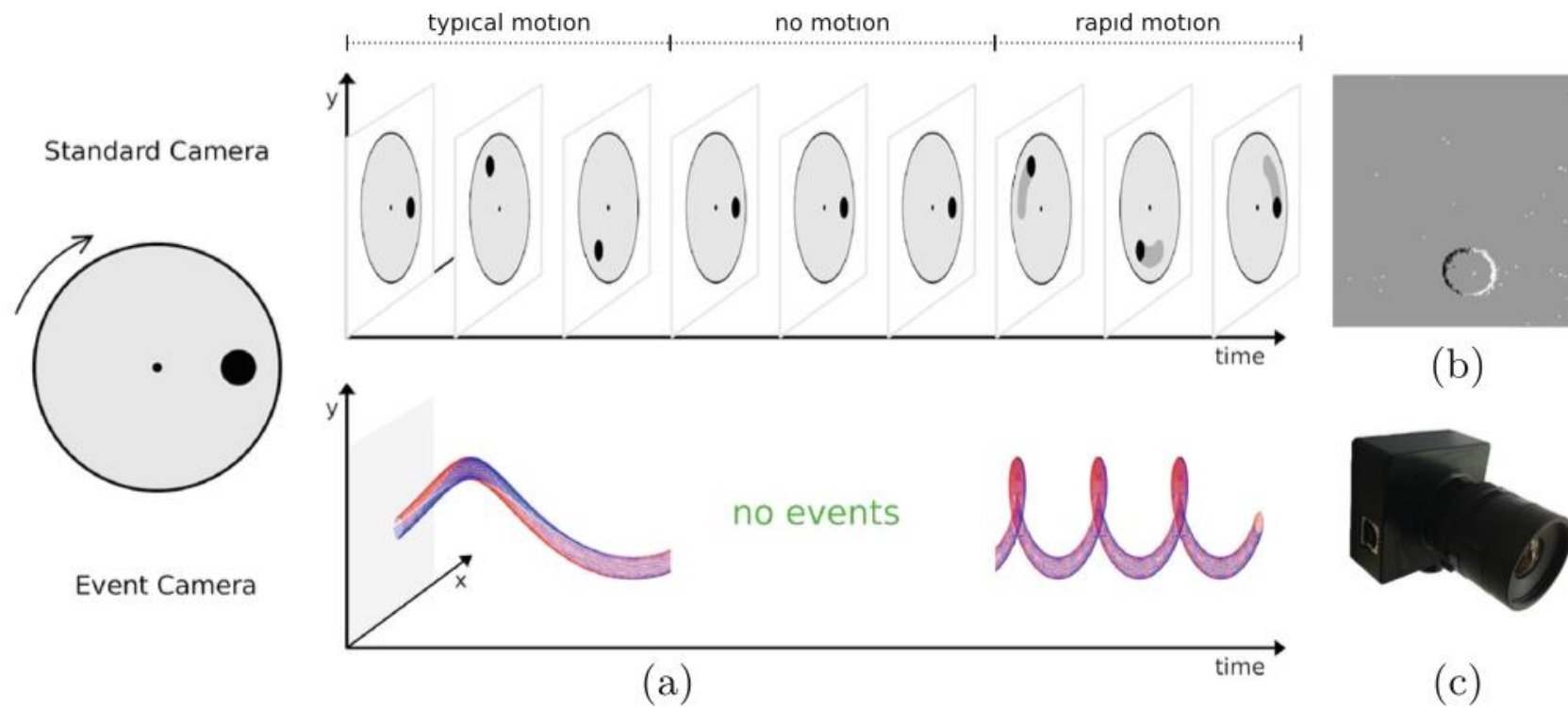


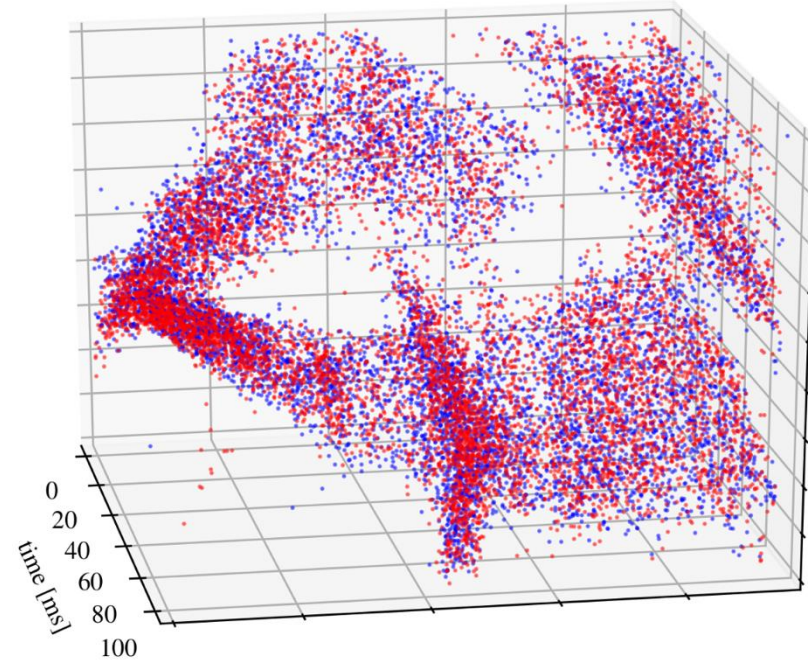
Diagram from the paper



Event cameras vs regular cameras for SLAM

- Event camera has high temporal contrast
 - Conventional imaging sensors have high latency (motion between frames)
- Event camera has high dynamic range
 - Conventional imaging sensors have low dynamic range (noise when image contains very bright and dark regions)
- With high temporal contrast and high dynamic range, SLAM has more information about how the scene is changing, which in turn means more signal for estimating structure and motion.

So suppose we use an event camera and capture events in our scene.



How do we use this output to conduct SLAM?

Event cameras for SLAM

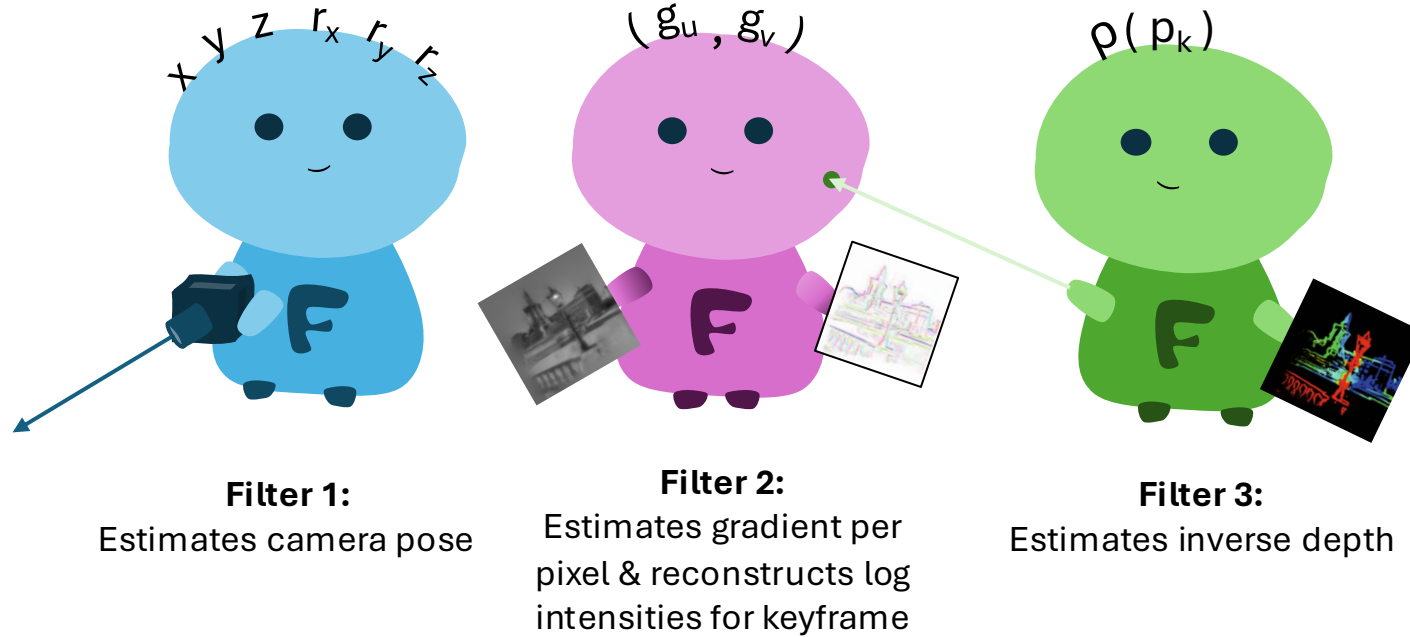
- SLAM = localization + mapping
- How to use output from event cameras to localize and map?
 - Localization = camera pose of event camera
 - But what about mapping?
 - From a stream of events alone, you cannot really figure out depth and therefore the overall 3D structure of the environment
 - Event cameras don't provide per-pixel intensity information that would be needed for conventional 3D reconstruction
 - Therefore, authors divide mapping into two parts: per-pixel gradient (g_u, g_v) and inverse depth

Event cameras for SLAM

- For SLAM, we have to figure out three kinds of information from the raw stream of events:
 - 1. camera pose
 - 2. gradient
 - 3. inverse depth
- To figure these out, paper uses three probabilistic filters, each filter predicting one component

The three filter fairies that SLAM

We trust each other!



These filters work together to do SLAM.

They provide their own outputs for other filters to use.

They use each other's output to compute their own outputs.

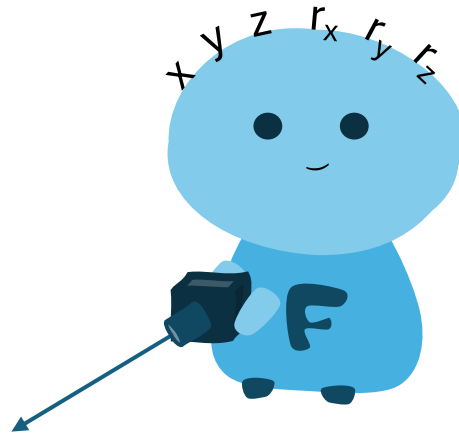
Each filter, therefore, assumes that the **information provided by the other filter is always correct.**

Methods: Preliminaries

- The paper denotes each event as $e(u, v) = (u, v, p, t)^\top$
 - u, v = pixel location
 - p = polarity (+1 for positive event, -1 for negative event)
 - t = microsecond-precise timestamp
- τ_c = time elapsed since the previous event
- In addition:
 - Event camera has **fixed pre-calibrated intrinsic matrix K**
 - All event pixel locations **prewarped** to remove radial distortion

Methods, pt. 1: Tracking 6-DoF Event Camera

Introducing Filter 1



Mission: to predict camera motion and accordingly update pose measurement

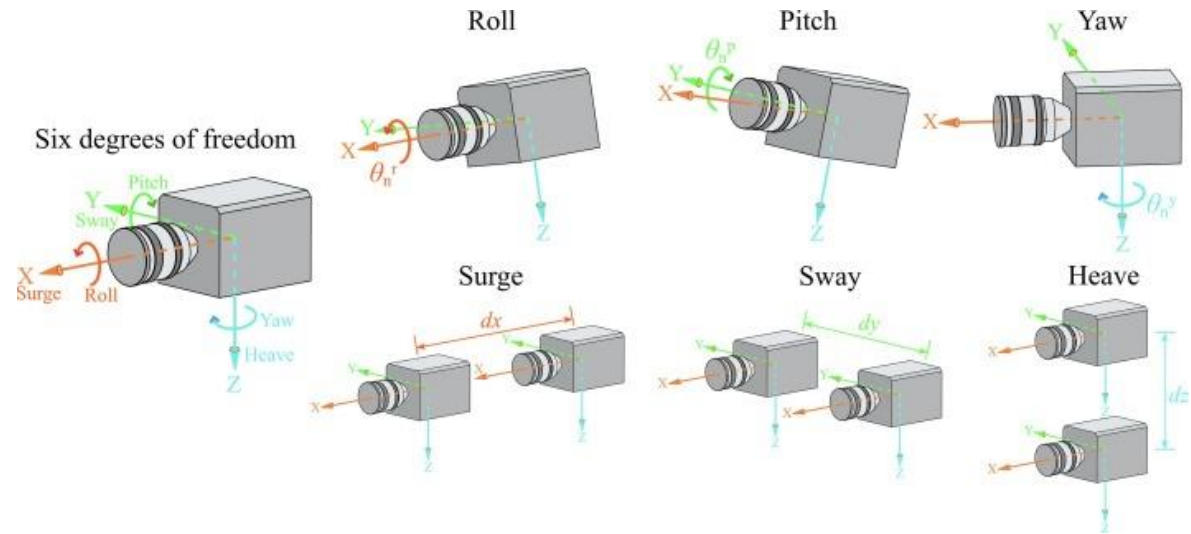
Main tool: Extended Kalman Filter

Additional resources:

- Filter 2's gradient and intensity information
- Filter 3's depth information

Methods, pt. 1: Describing Motion

- Camera motion described by six values:
 - Translation
 - Rotation



Methods, pt. 1: Working with a valid camera pose

- State vector $\mathbf{x} \in \mathbb{R}^6$ is used to represent camera pose c with respect to the world frame of reference w

Methods, pt. 1: Working with a valid camera pose

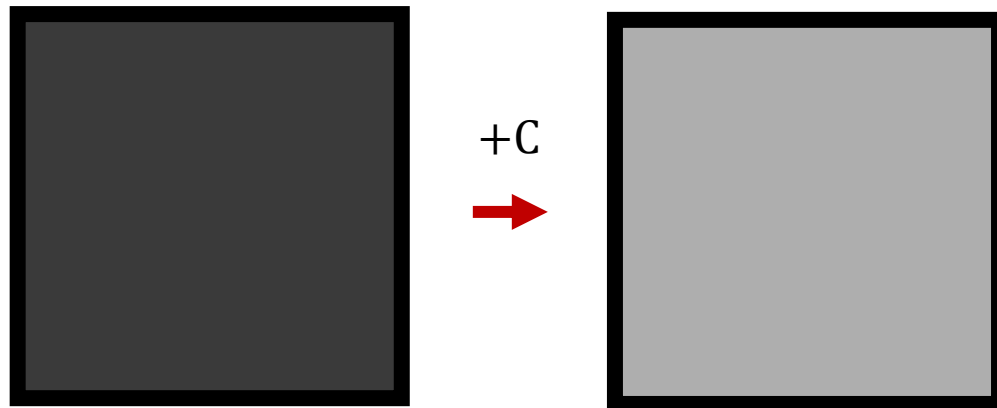
- State vector $\mathbf{x} \in \mathbb{R}^6$ is used to represent camera pose c with respect to the world frame of reference w
- To ensure camera pose translations and rotations follow laws of physics, vector $\mathbf{x}$ is mapped to a member of the Lie Group SE(3)
 - SE(3) = catalog of all 3D rigid body transformations;
 - Lie group = subset of SE(3) where poses are smooth and continuous
 - By forcing the camera pose to be a member of the SE(3), we ensure that the camera stays a solid object even as if we apply translations and rotations

Methods, pt. 1: Keeping track of uncertainty along with camera pose

- Each vector $\mathbf{x}$ is also accompanied by a covariance matrix $\mathbf{P}_{\mathbf{x}} \in \mathbb{R}^{6 \times 6}$

Methods, pt. 1: Setup

At time $t - \tau_c$, event camera detects an event at pixel (u, v)

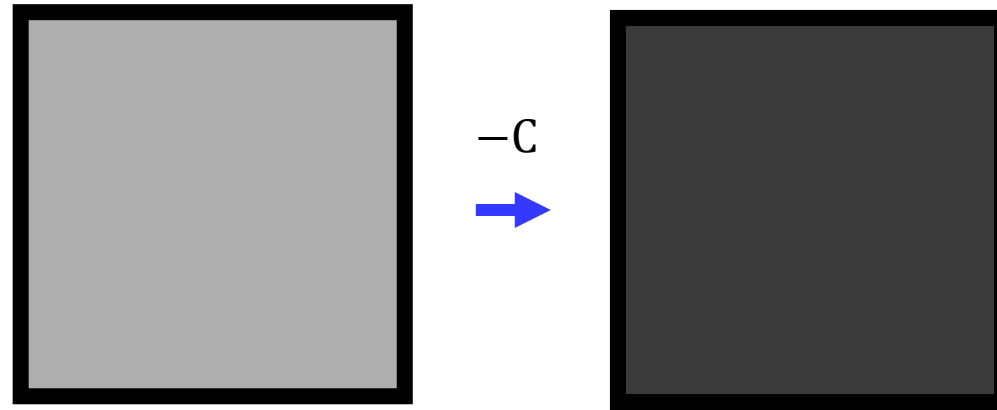


The camera pose is $\mathbf{T}_{wc}^{(t-\tau_c)}$

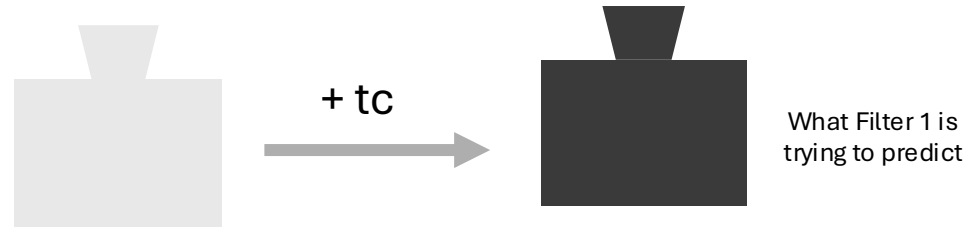


Methods, pt. 1: Setup

At time t , event camera detects another event at pixel (u, v)



The camera pose is now $\mathbf{T}_{wc}^{(t)}$



What information does filter 1 have available?

- Camera pose at time $t - \tau_c$ (from previous estimate)
- Difference in intensity between both camera poses is $\pm C$
 - *Note:* Filter 1 does not know intensity at pixel (u, v) ; just that it underwent either a positive or negative change in intensity described by $\pm C$
- Filter 2's information about pixel (u, v) :
 - gradient at (u, v) and the reconstructed intensity at that pixel
- Filter 3's information about pixel (u, v) :
 - inverse depth at (u, v)

Methods, pt. 1: How does filter 1 estimate 6 DoF Event camera?

- Two steps to tracking 6-DoF event camera:
 - A. Predicting motion (predicting how camera pose changed between two events)
 - B. Updating camera pose measurement accordingly

Methods, pt. 1A: Motion prediction

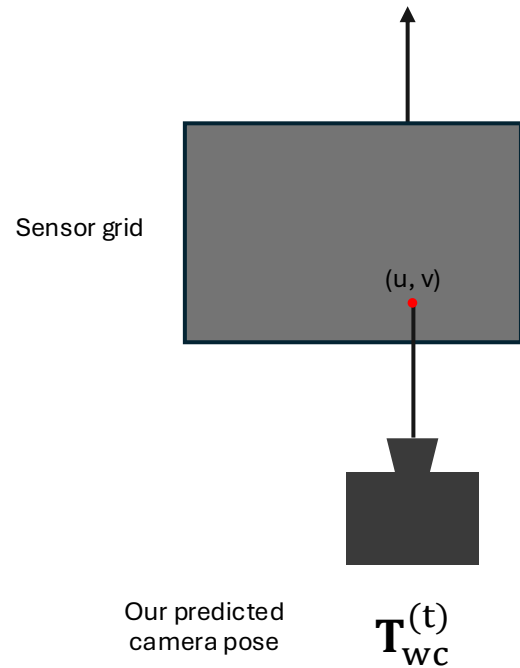
- Constant position motion model
 - Assumption that the camera pose did not change between previous event and current event (gaps between events are normally microseconds long)

$$\mathbf{x}^{(t|t-\tau)} = \mathbf{x}^{(t-\tau|t-\tau)} + \mathbf{n}$$

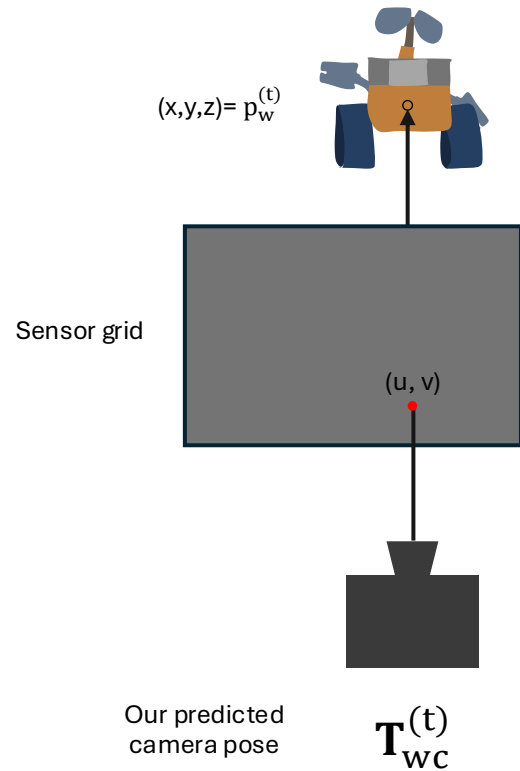
- We also update our covariance matrix to take note of the increase in error our prediction can cause

$$\mathbf{P}_{\mathbf{x}}^{(t|t-\tau)} = \mathbf{P}_{\mathbf{x}}^{(t-\tau|t-\tau)} + \mathbf{P}_{\mathbf{n}}$$

Methods, pt. 1B: Measurement update

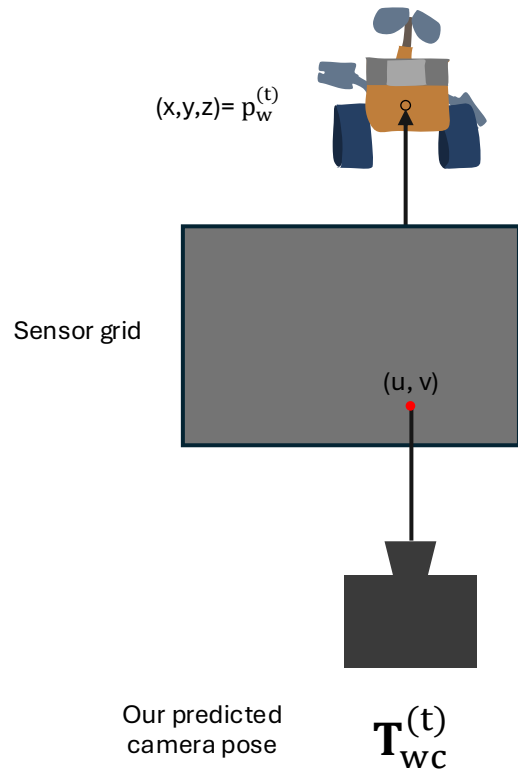


Methods, pt. 1B: Measurement update

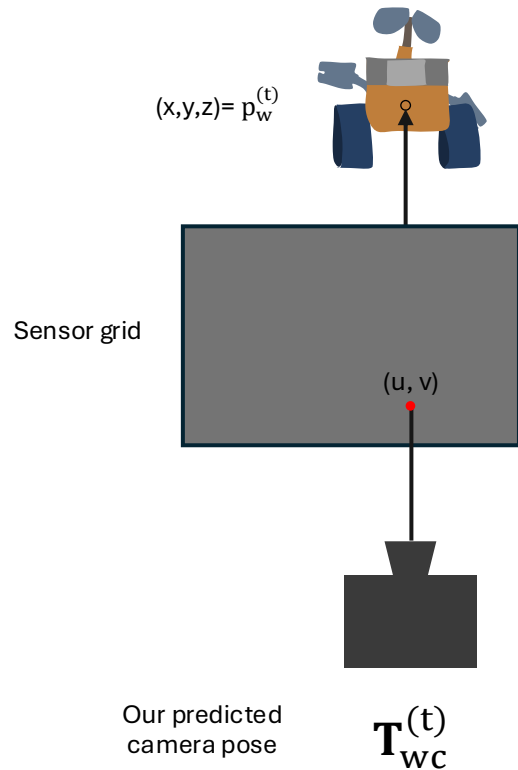


Methods, pt. 1B: Measurement update

How does it retrieve 3D scene point?

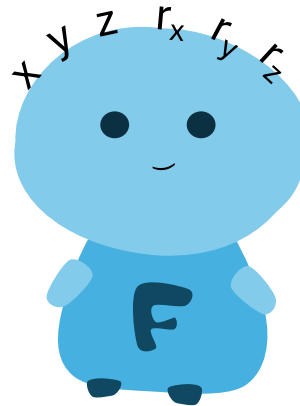


Methods, pt. 1B: Measurement update

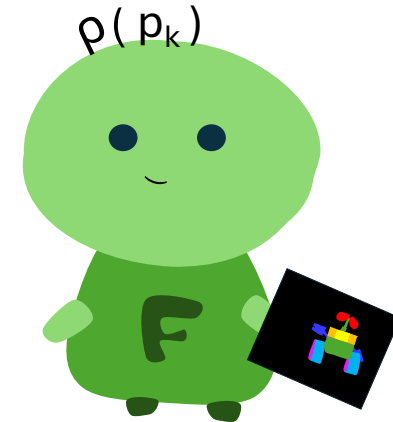


How does it retrieve 3D scene point?

High level

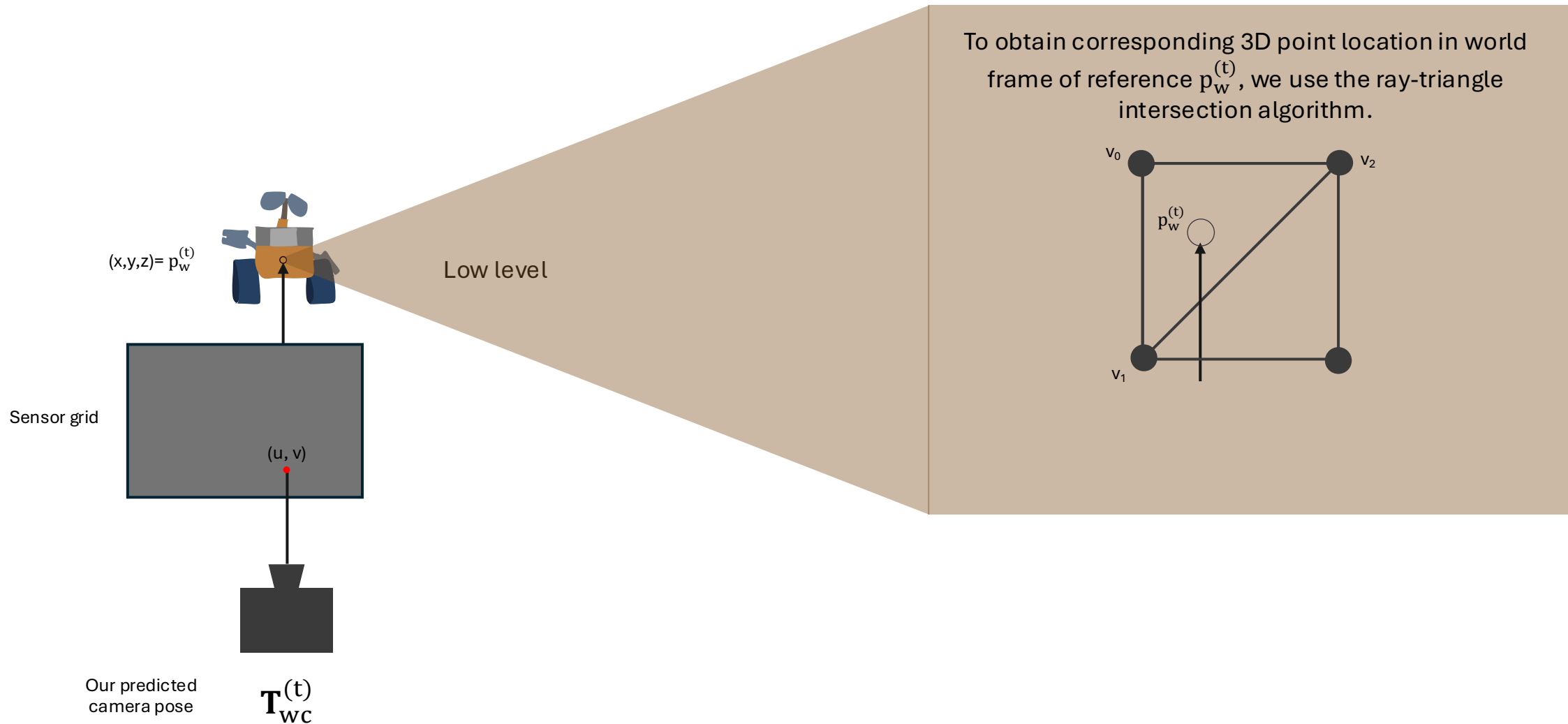


What's the 3D coordinate of pixel (u, v) from this camera position?

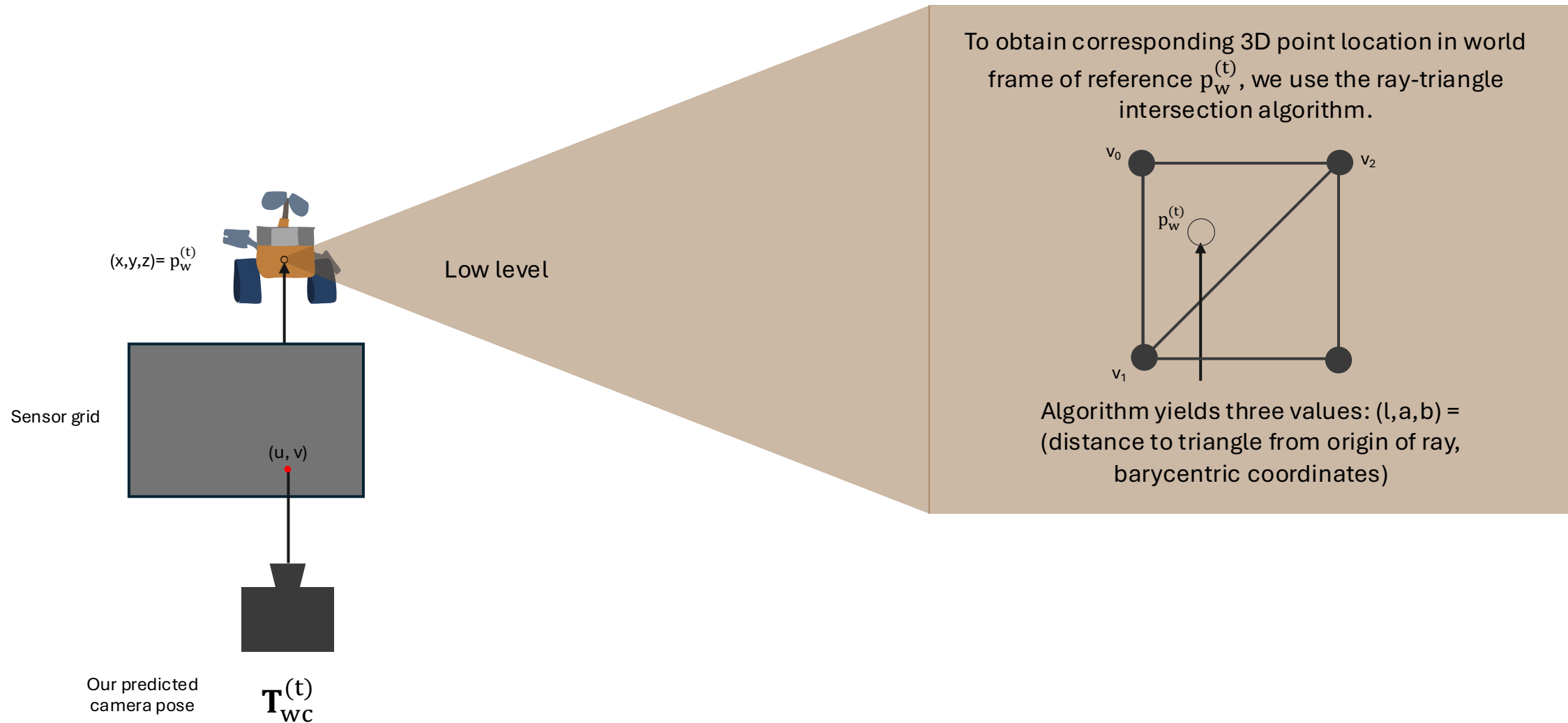


Based on my current 2D depth map and 3D point cloud, the 3D coordinate is $p_w^{(t)}$!

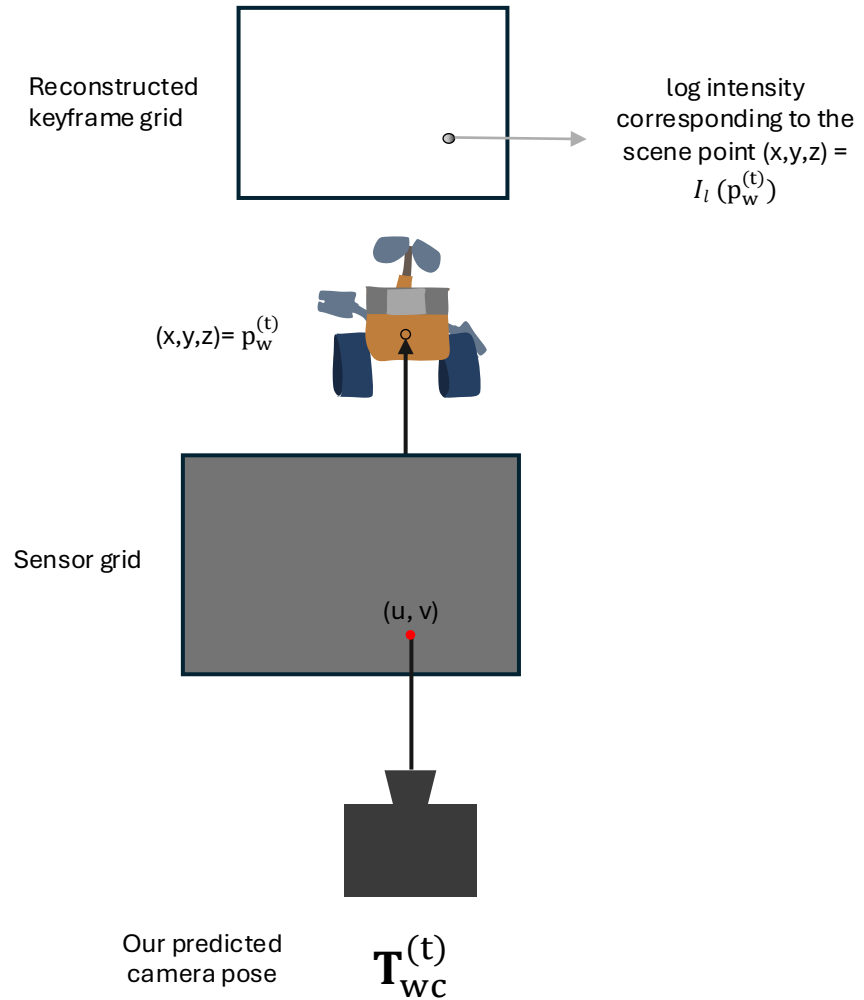
Methods, pt. 1B: Measurement update



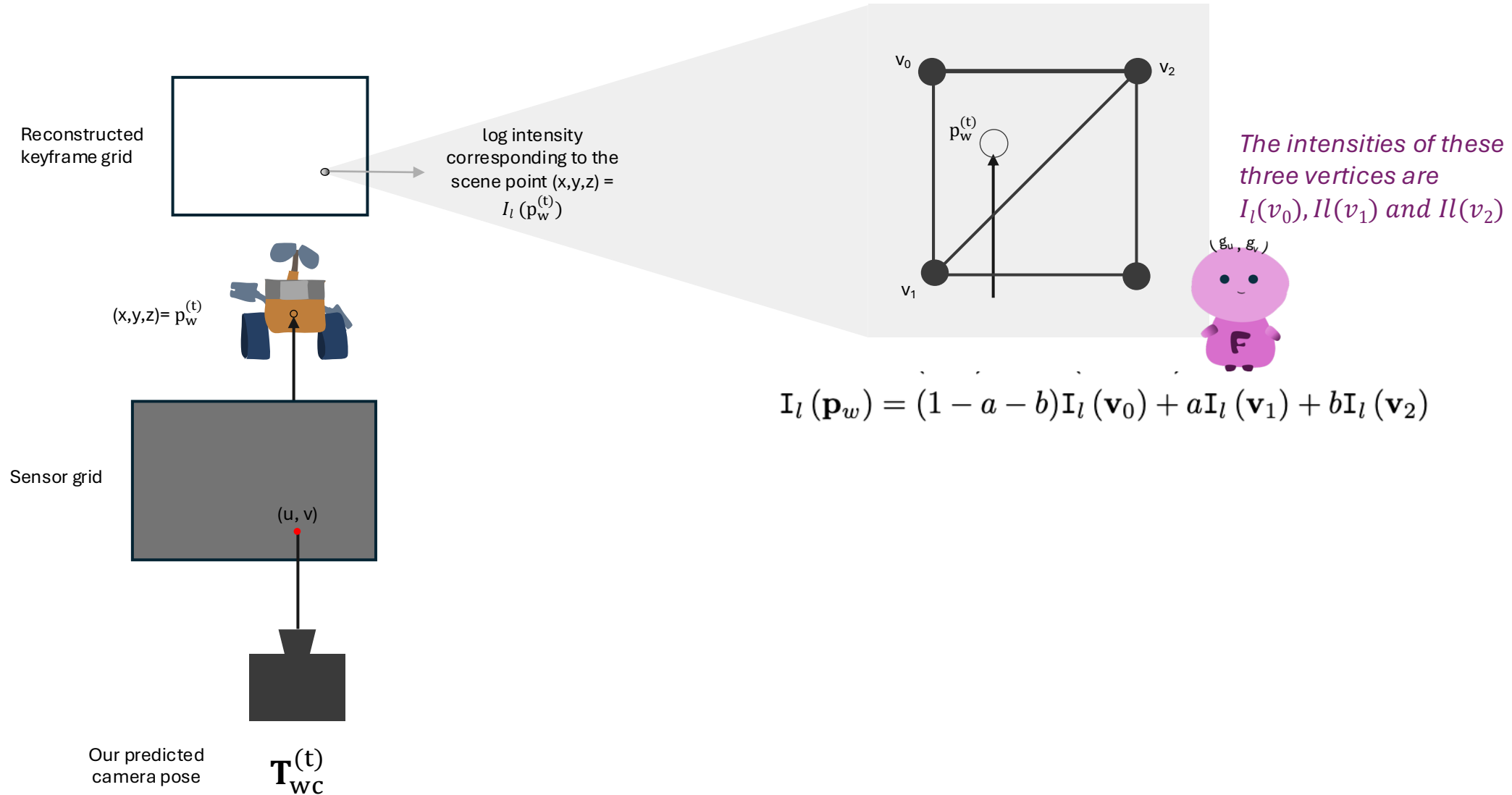
Methods, pt. 1B: Measurement update



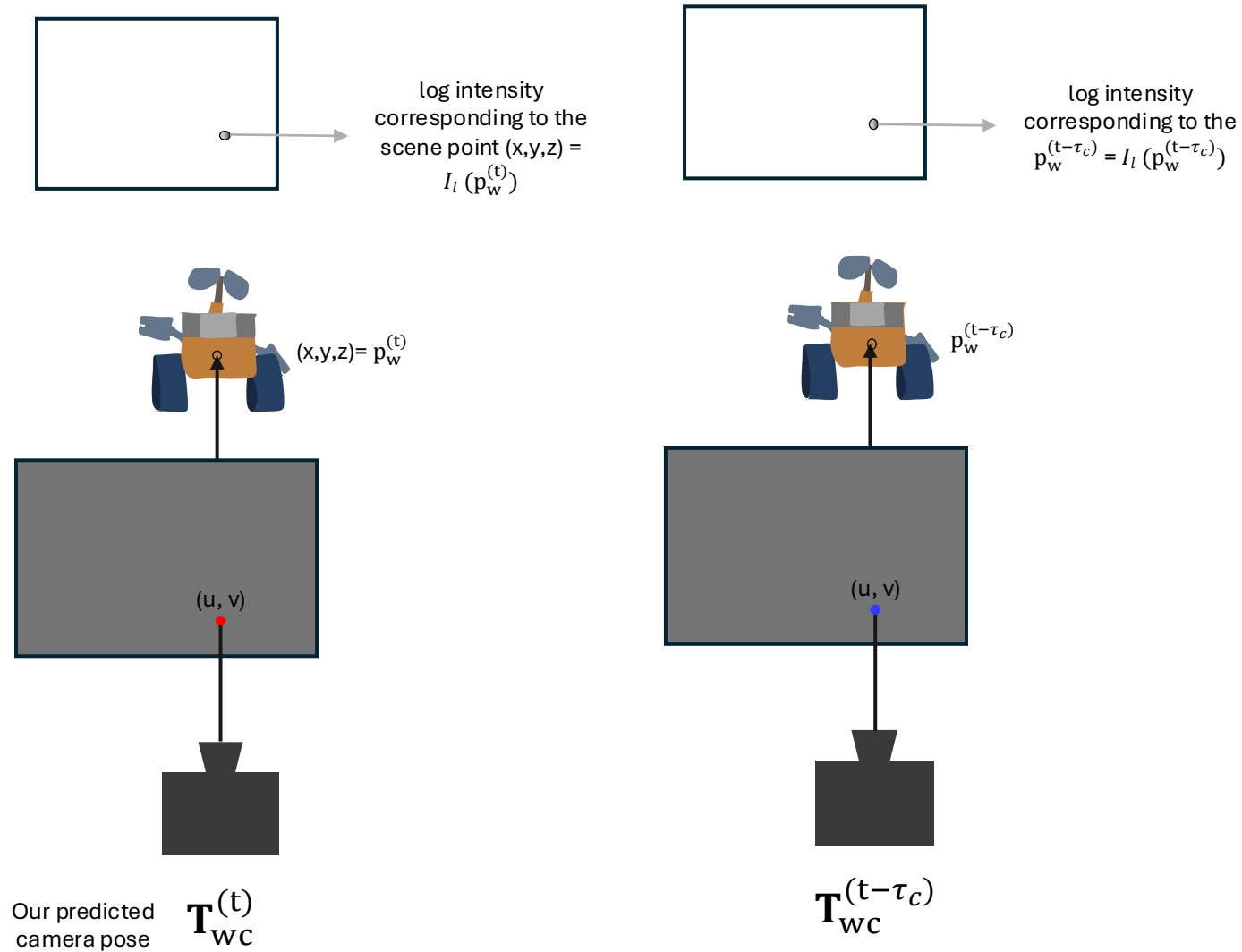
Methods, pt. 1B: Measurement update



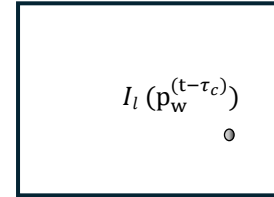
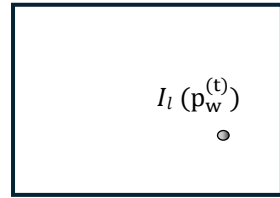
Methods, pt. 1B: Measurement update



Methods, pt. 1B: Measurement update



Methods, pt. 1B: Measurement update



Filter 1's predicted measure of log intensity difference between both points

$$h_{\mathbf{x}}(\mathbf{x}^{(t|t-\tau)}) = I_l(\mathbf{p}_w^{(t)}) - I_l(\mathbf{p}_w^{(t-\tau_c)})$$

Actual measurement (how much intensity actually changed at pixel (u,v) between time t and t-tc):

$$z_{\mathbf{x}} = \pm C$$

Compute innovation:

If innovation is zero, the current pose estimate perfectly explains the event.

If it's nonzero, the pose estimate is wrong by some amount in some direction and needs updating.

Methods, pt. 1B: Measurement update

To find how to update measurements, filter 1 uses Kalman gain $\mathbf{W}_x$, which converts that scalar innovation into a correction across all six pose dimensions and figures out which combination of position and rotation adjustments would best explain the observed gap.

The pose estimate gets nudged by that correction and the uncertainty shrinks.

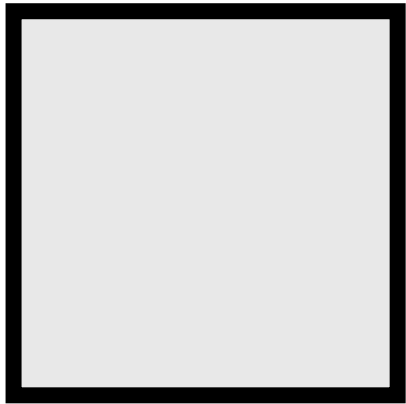
$$\mathbf{x}^{(t|t)} = \mathbf{x}^{(t|t-\tau)} + \mathbf{W}_x \nu_x,$$
$$\mathbf{P}_x^{(t|t)} = \left(\mathbf{I}_{6 \times 6} - \mathbf{W}_x \frac{\partial h_x}{\partial \mathbf{x}^{(t|t-\tau)}} \right) \mathbf{P}_x^{(t|t-\tau)},$$

Overall idea:

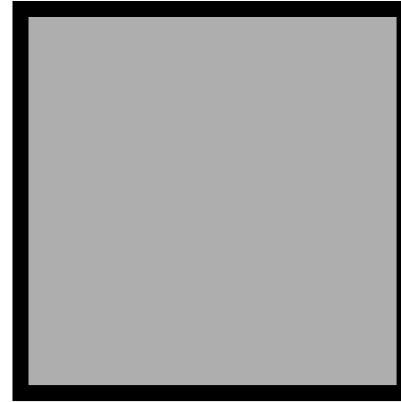
Filter 1 tries to find the camera pose that best explains the change in intensities between both events

Predicted trajectory:

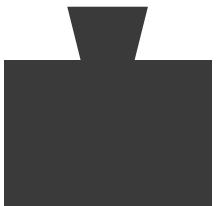
Wrong camera pose estimate gives us different intensity change



+A



-C



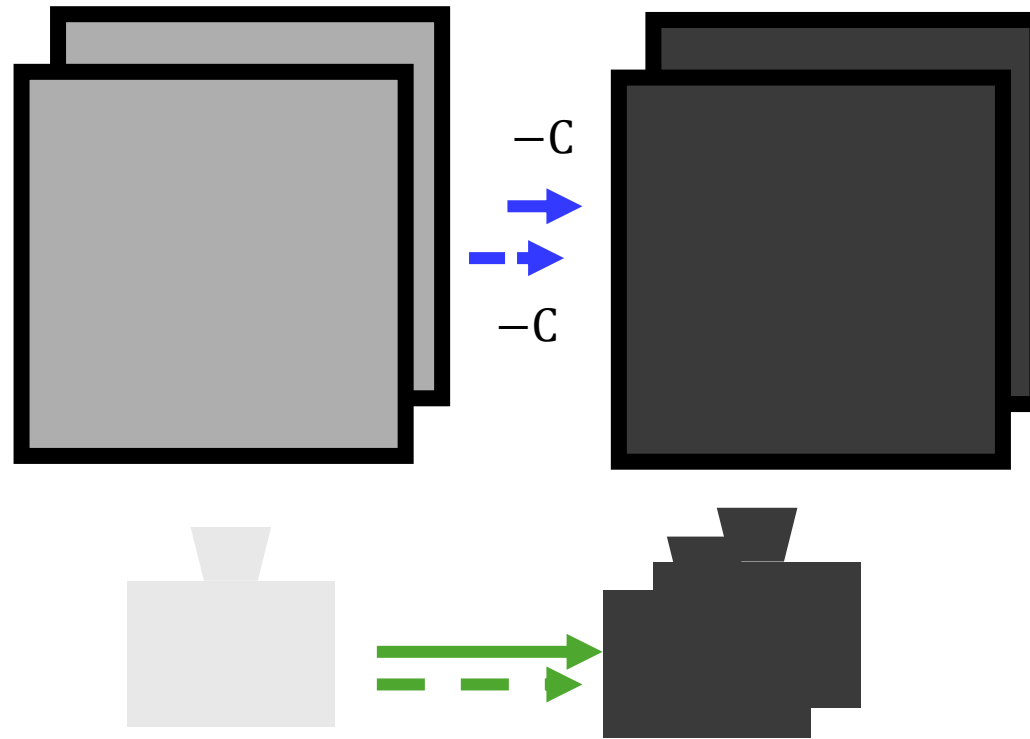
What Filter 1 is trying to predict

Wrong camera pose $\mathbf{T}_{wc}^{(t)}$

Actual camera pose $\mathbf{T}_{wc}^{(t)}$

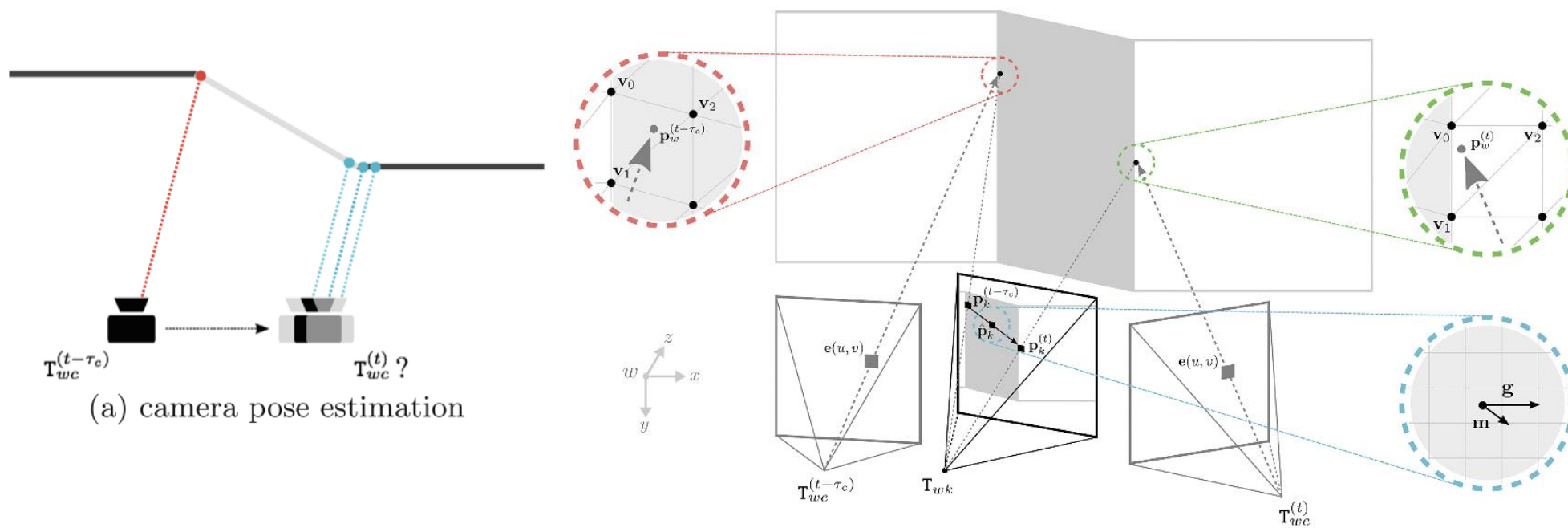
Overall idea:

Filter 1 tries to find the camera pose that best explains the change in intensities between both events



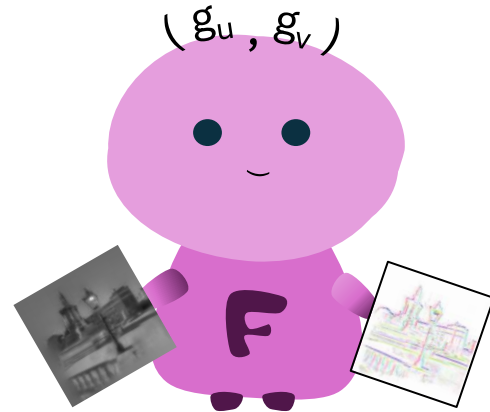
If our predicted trajectory is correct, it would resemble true trajectory.

Diagrams from the paper



Methods, pt. 2: Gradient Estimation and Log Intensity Reconstruction

Introducing Filter 2



Mission: to predict per-pixel gradient and use that information to reconstruct log intensity per keyframe pixel

Main tool: Extended Kalman Filter (used per-pixel)

Additional resources:

- Filter 1's camera pose
- Filter 3's per-pixel depth

Methods, pt. 2A: What are we trying to predict?

- Gradient for pixel (u, v) :
 - Described as (g_u, g_v) , which represents the horizontal and vertical components of the magnitude and direction of the steepest intensity change for pixel (u, v)
 - Similar to filter 1, this estimate comes with a covariance matrix

Methods, pt. 2A: How do we predict gradient?

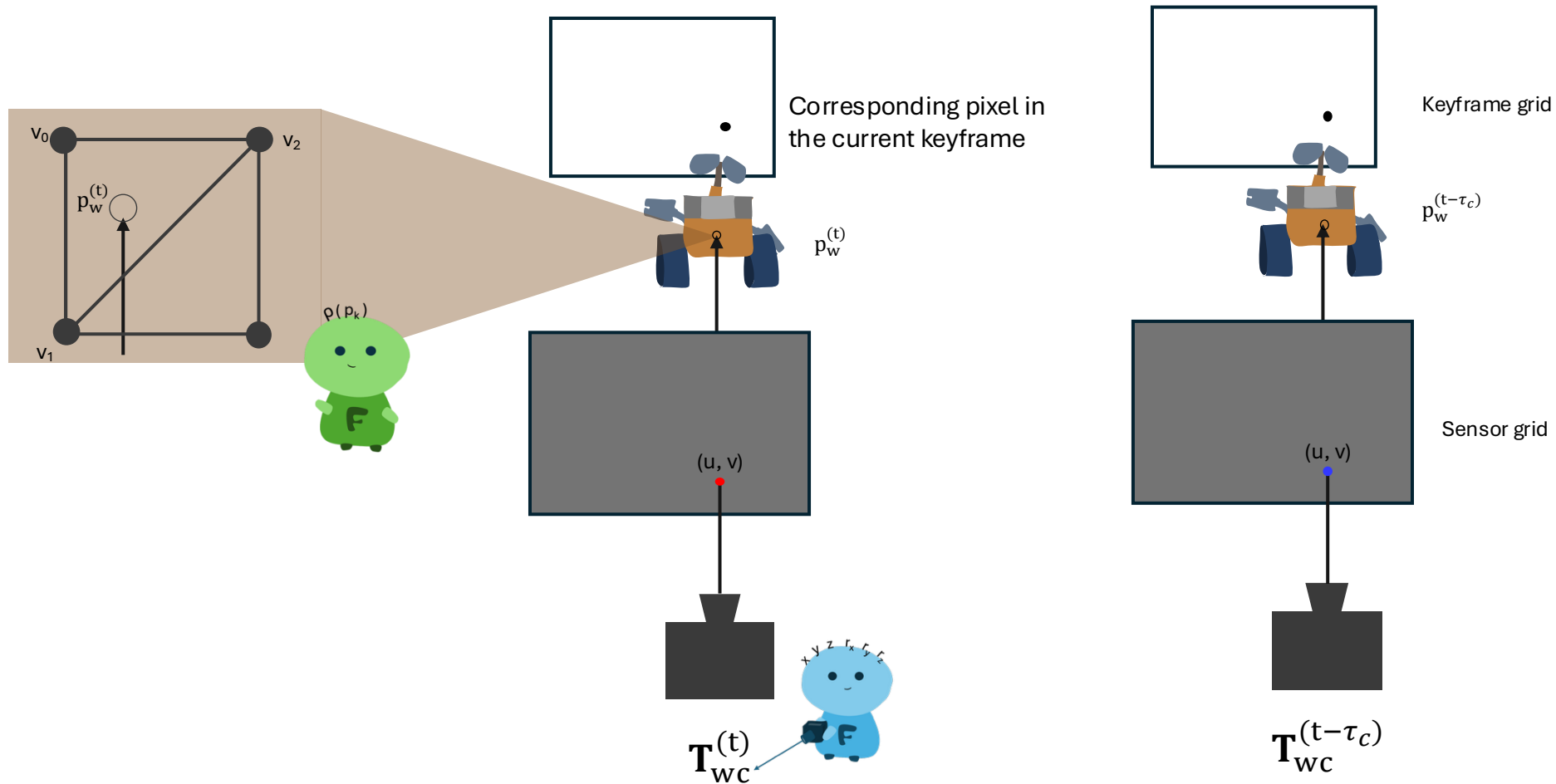
- **Brightness constancy equation:**
 - Assumes that as camera moves, the brightness of a scene point doesn't change, only its position in the image changes
- The paper uses this equation to derive the following:
 - $\text{gradient} \cdot \text{motion} = \text{predicted brightness change}$
- Based on this mathematical assumption, filter 2...
 - Has some **actual observed** brightness change rate
 - Represented as $\frac{\pm C}{\tau_c} = z_g$ (instantaneous brightness change at this pixel)
 - Compares it with its **predicted** brightness change rate
 - Represented as $\text{gradient} \cdot \text{motion vector} = h_g$

Methods, pt. 2A: How do we predict gradient?

- In order to compute our predicted measurement, we need to find:
 - g = the gradient estimate at some point at time $t - \tau_c$
 - Initially, all gradients are initialized to zero with large variances
 - m = the motion vector = $\frac{\text{displacement between two corresponding pixels}}{\tau_c}$

Methods, pt. 2A: How to find motion?

- m = motion vector = $\frac{\text{displacement between two corresponding pixels}}{\tau_c}$



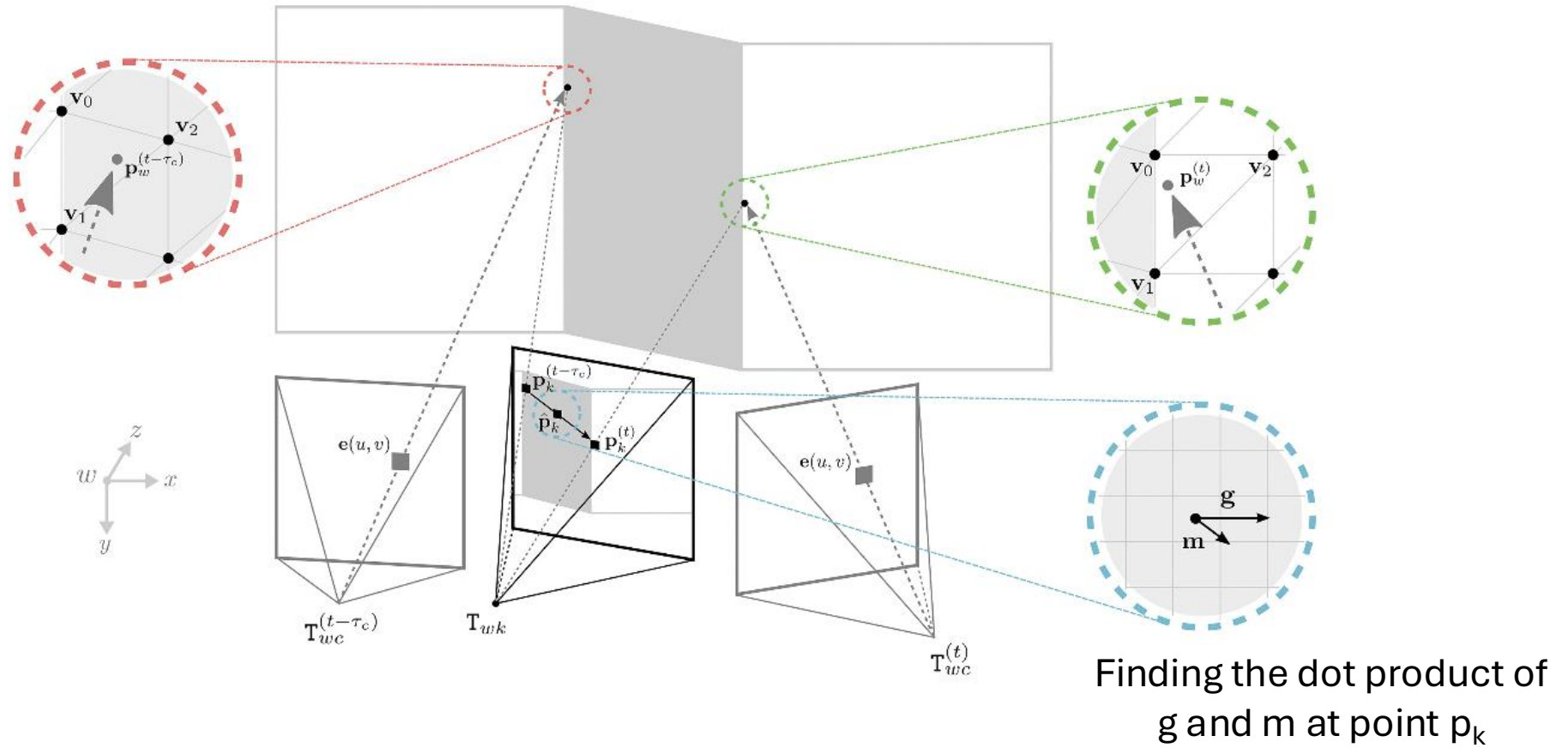
Methods, pt. 2A: Updating gradient

- We compare the difference between $z_g = \frac{\pm C}{\tau_c}$ and $h_g = g \cdot m$
- Using innovation and standard EFK procedures, we use Kalman gain to update the gradient (g_u, g_v) and reduce the uncertainty in covariance matrix accordingly

Methods, pt. 2A: Where to apply gradient update?

- We have two locations available $p_w^{(t)}$ & $p_w^{(t-\tau_c)}$.
- But the observed brightness change rate happened over the entire time interval between $(t - \tau_c)$ & t
- Therefore, we apply the gradient update to midpoint $\hat{p}_k$
 - Most representative single location as it sits at the center of the region over which the brightness change accumulated

Diagram from the paper



Methods, pt. 2B: Log Intensity Reconstruction

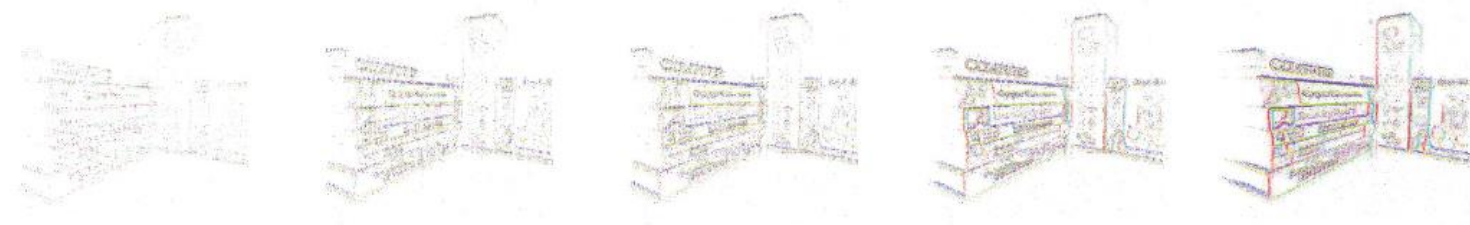
- Filter 2 also integrates the gradient map to recover actual brightness values at every keyframe pixel.
 - This runs in parallel with gradient estimation on the GPU.
- The following minimization function is used:

$$\min_{\mathbf{I}_l} \left\{ \int_{\Omega} \|\mathbf{g}(\mathbf{p}_k) - \nabla \mathbf{I}_l(\mathbf{p}_k)\|_{\epsilon_d}^h + \lambda \|\nabla \mathbf{I}_l(\mathbf{p}_k)\|_{\epsilon_r}^h d\mathbf{p}_k \right\}$$

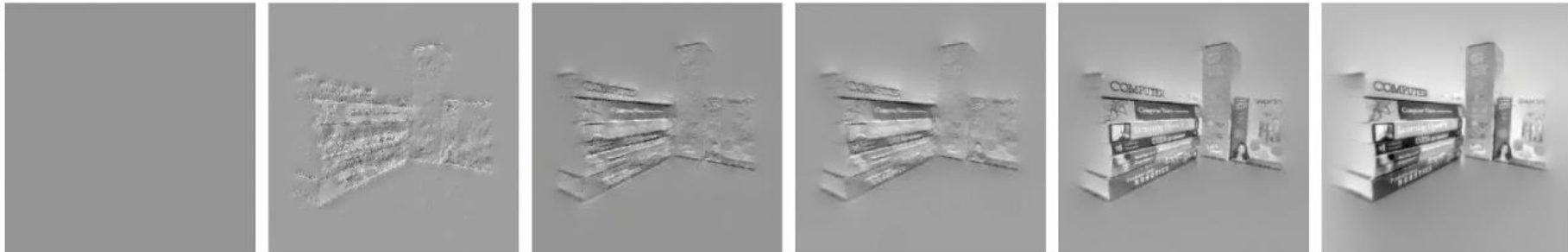
- Data term $\|g(p_k) - \nabla I_l(p_k)\|_d$ = how well the reconstructed image's own gradients ∇I_l match the estimated gradient map g
- Regularisation term $\lambda \|\nabla I_l(p_k)\|_r$ = penalises large gradients in the reconstruction, gently pushing it toward smoother solutions

Diagram from the paper

Gradient
estimation



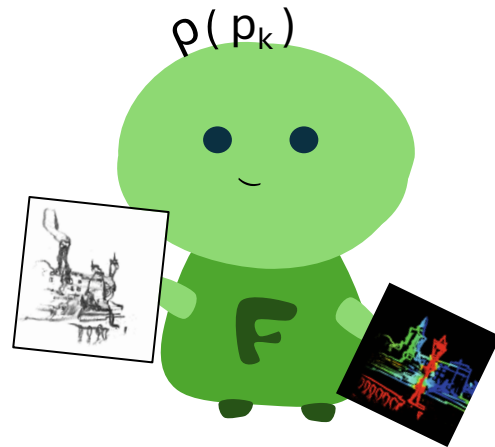
Log intensity
reconstruction



Shows the typical temporal progression of gradient estimation and log intensity reconstruction as the camera explores the 3D scene

Methods, pt. 3: Inverse Depth Estimation and Regularisation

Introducing Filter 3



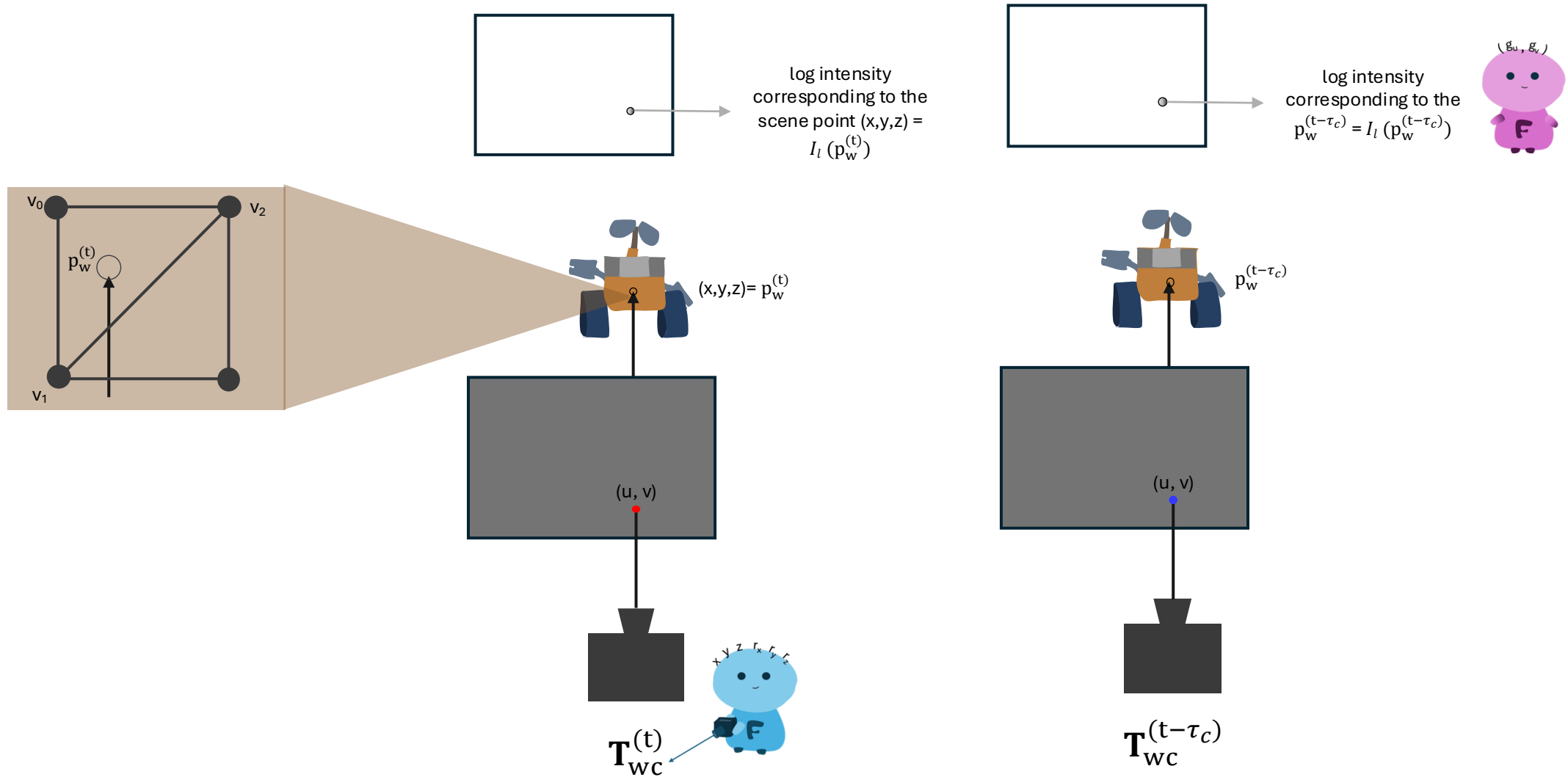
Mission: to predict per-pixel inverse depth

Main tool: Extended Kalman Filter (used per-pixel)

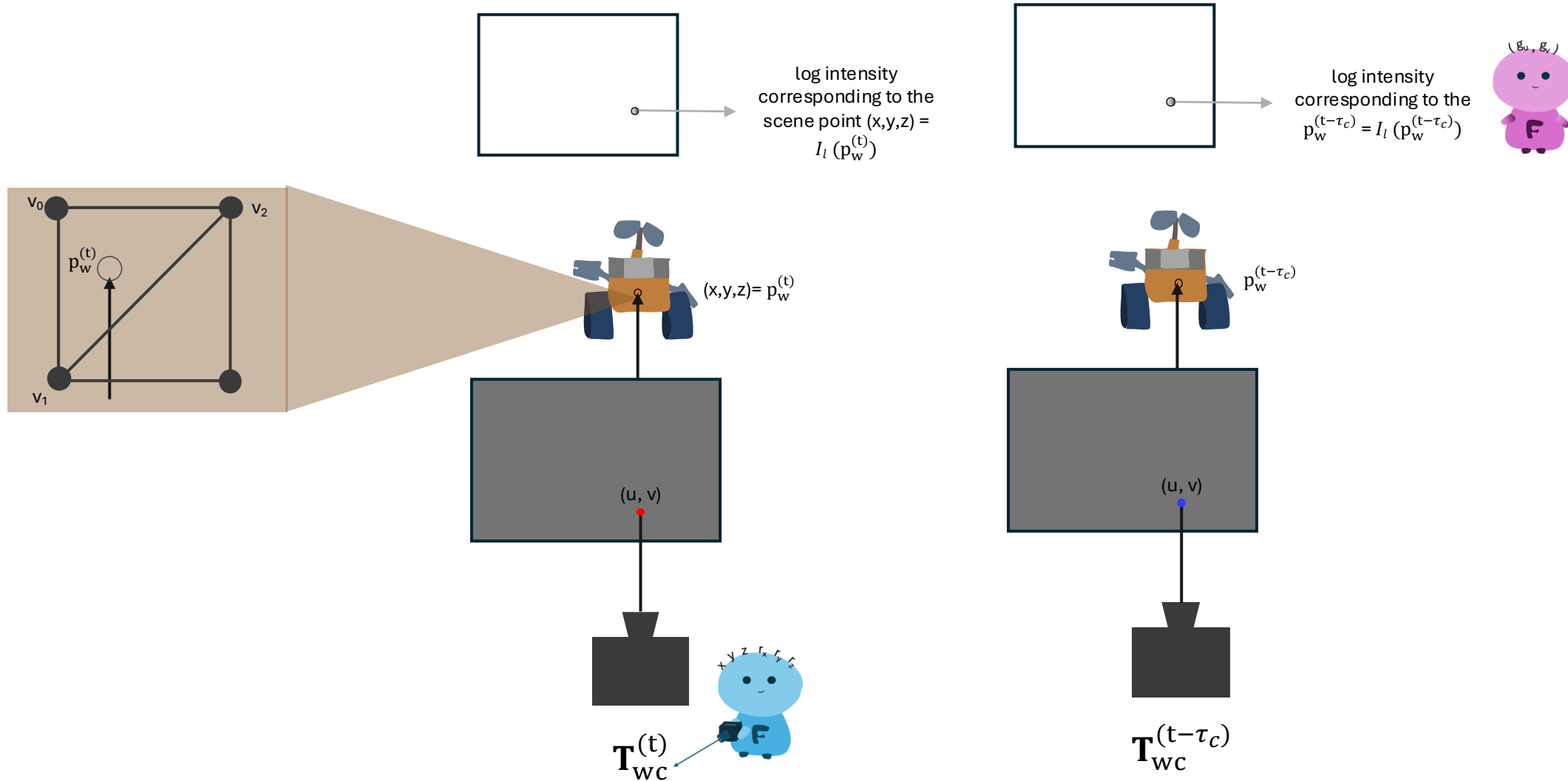
Additional resources:

- Filter 1's camera pose
- Filter 2's reconstructed log intensity for keyframe

How to predict inverse depth?



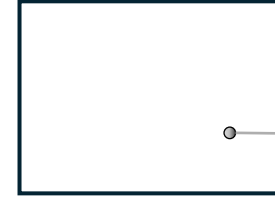
What's different from filter 1's procedure?



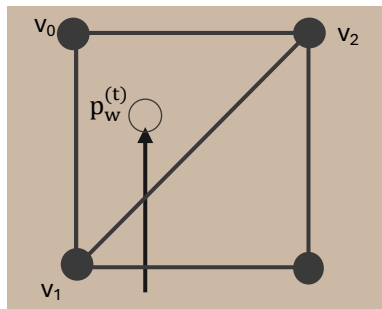
What's different from filter 1's procedure?



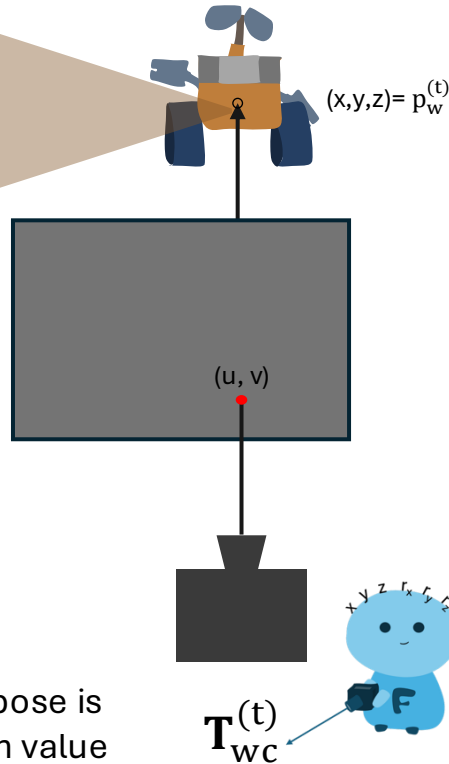
log intensity
corresponding to the
scene point $(x,y,z) =$
 $I_l(p_w^{(t)})$



log intensity
corresponding to the
 $p_w^{(t-\tau_c)} = I_l(p_w^{(t-\tau_c)})$

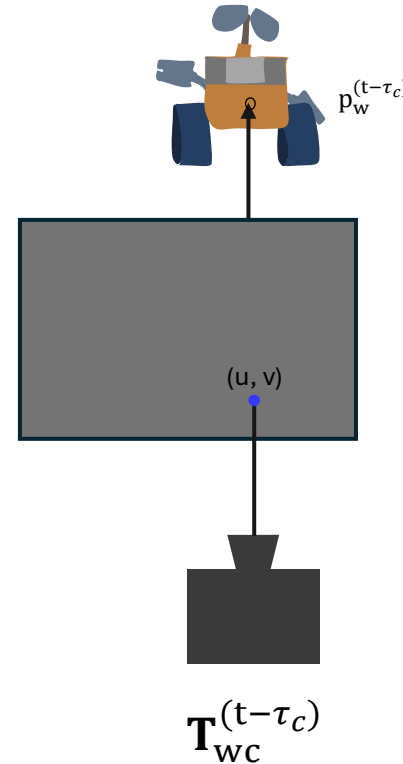


The inverse depths are not
treated as known values



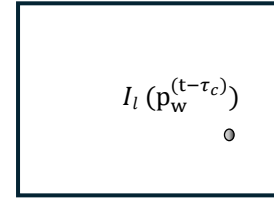
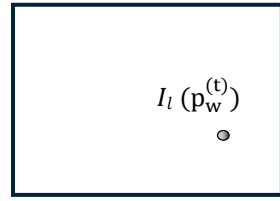
Now the camera pose is
treated as a known value

$T_{wc}^{(t)}$



$T_{wc}^{(t-\tau_c)}$

How to predict inverse depth (cont.)?



Filter 3's predicted measure of log intensity difference between both points

$$h_{\mathbf{x}}(\mathbf{x}^{(t|t-\tau)}) = \mathcal{I}_l(\mathbf{p}_w^{(t)}) - \mathcal{I}_l(\mathbf{p}_w^{(t-\tau_c)})$$

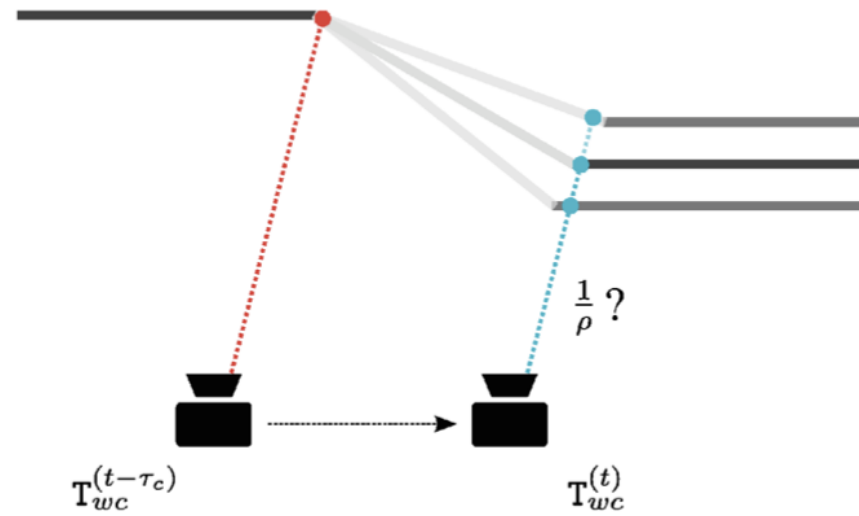
Actual measurement (how much intensity actually changed at pixel (u,v) between time t and t-tc):

$$z_{\mathbf{x}} = \pm C$$

Compute innovation (which compares between predicted and actual measurement)

Update depth estimate accordingly.

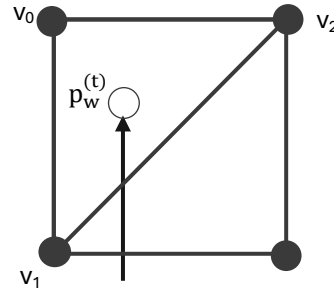
Diagram from paper



(b) inverse depth estimation

Updating EFK estimates

- Recall that the value of an inverse depth for a point depends on the three vertices it lies between

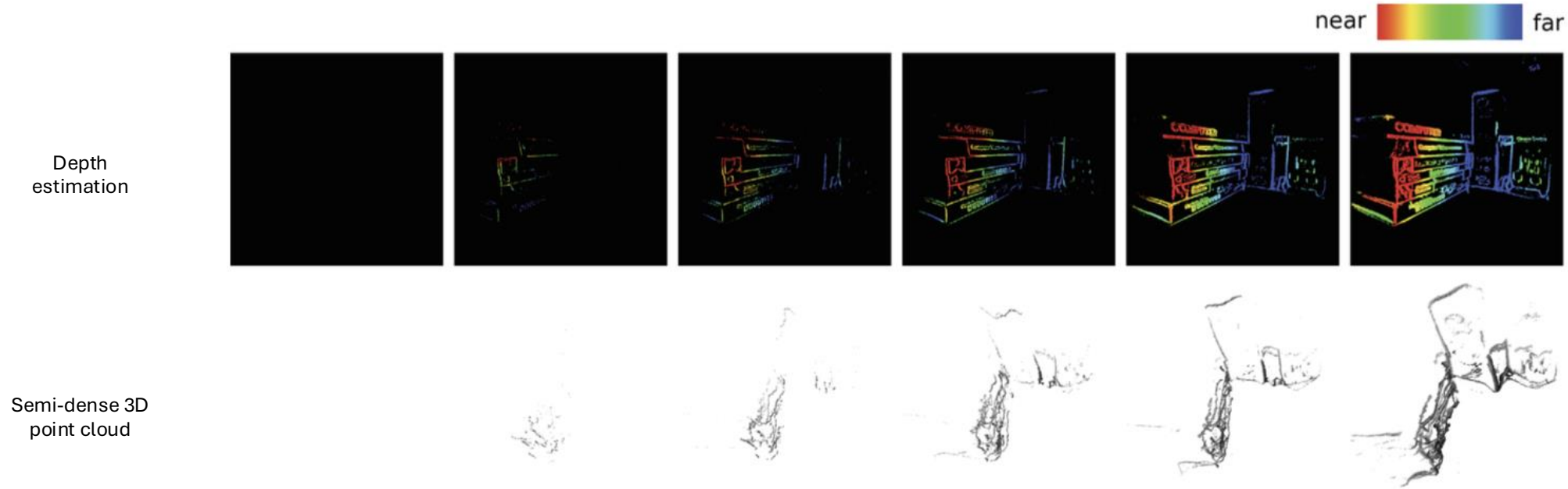


- When Filter 3 updates the estimate for a point's inverse depth following the innovation computation, it simultaneously updates the inverse depth values of all three vertices v_0 , v_1 , v_2 that contributed to the predicted measurement h_g
- Use Kalman gain to correct inverse depth values and shrink uncertainty in covariance matrix accordingly

Inverse depth regularisation

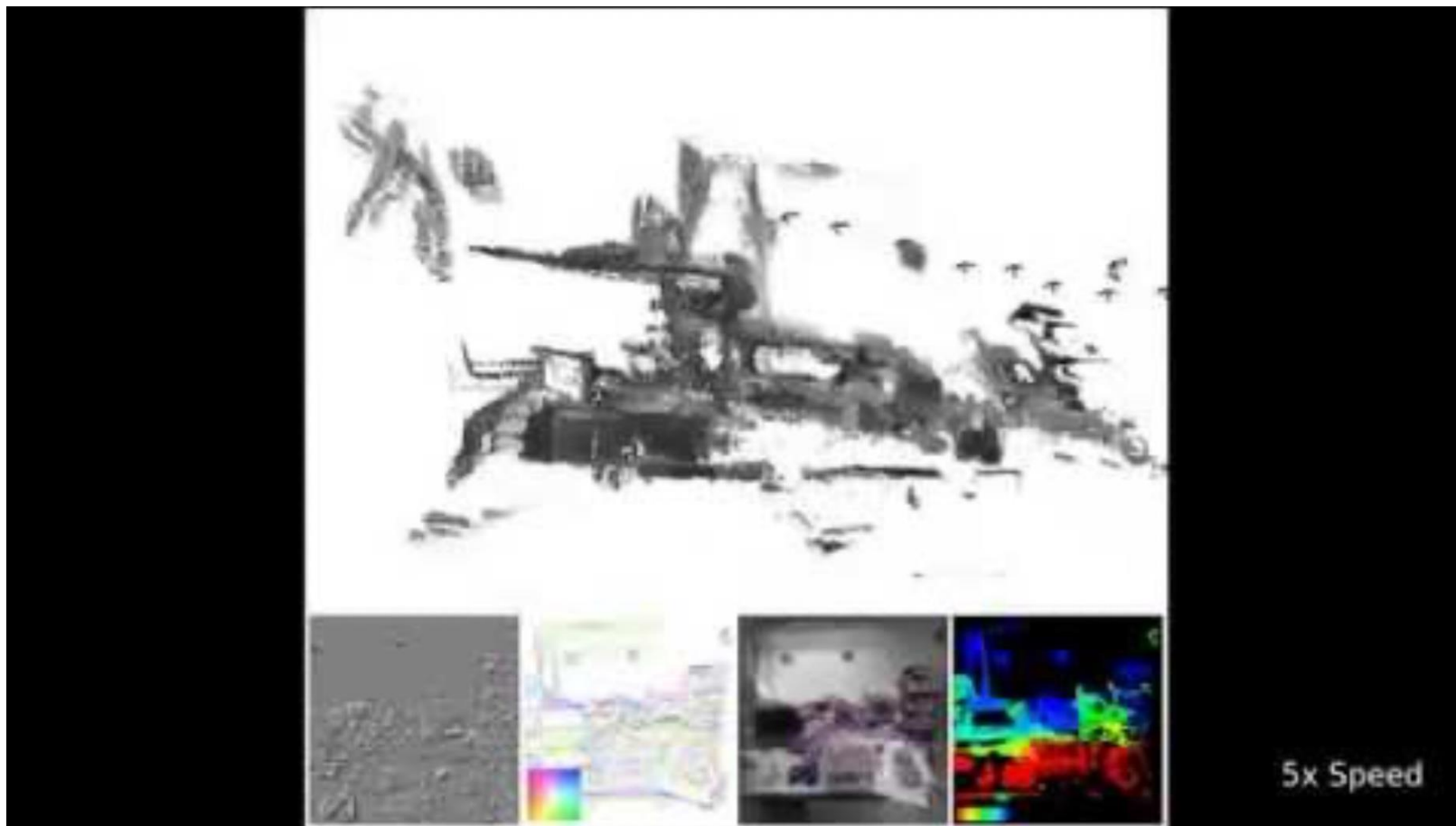
- As depth is being estimated, a regularisation step runs in the background where the depth map is continuously smoothened
- Regularisation is triggered when any pixel's depth estimates has changed significantly
- During a smoothing pass, each pixel gets nudged towards a weighted average of itself and its neighbors
 - Pixels with higher confidence (lower uncertainty) contribute higher to the average
 - Exception: neighbors differing by more than 2σ are excluded from the average to account for occlusion boundaries (i.e., regions where a nearby object's edge is in front of a distant background)

Diagram from the paper



Shows the typical temporal progression of inverse depth estimation and regularization as the event camera explores the 3D scene.

Experiments



Single Keyframe Estimation

- The system is able to conduct SLAM for a single keyframe
 - Even though camera moves around, it stays relatively close to its starting position



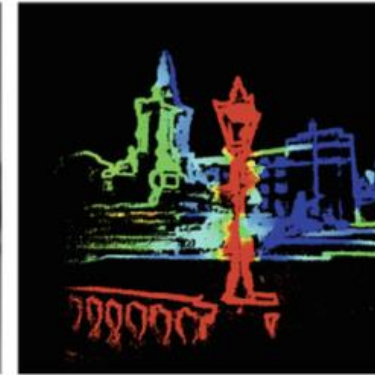
(a)
Visualisation of input
event stream



(b)
Estimated gradient
map for keyframe



(c)
Reconstructed
intensity keyframe



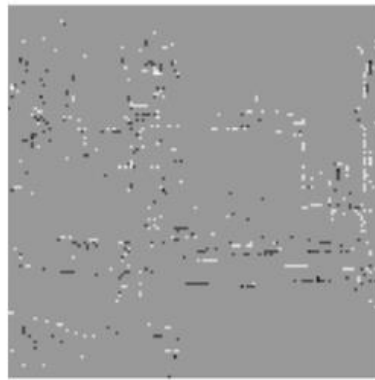
near  far

(d)
Estimated depth map
for keyframe



(e)
Semi-dense 3D point
cloud

Single Keyframe Estimation



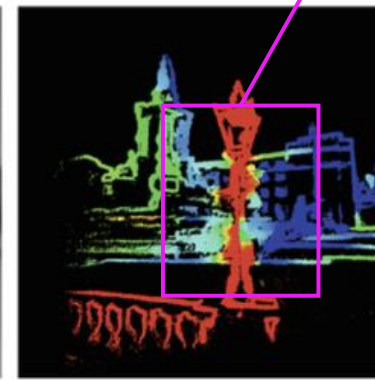
(a)
Visualisation of input
event stream



(b)
Estimated gradient
map for keyframe

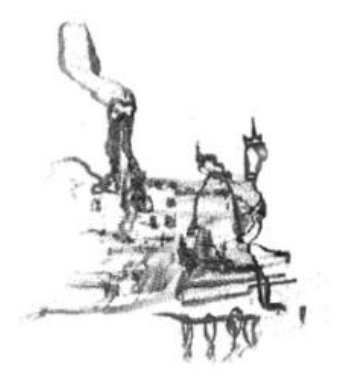


(c)
Reconstructed
intensity keyframe



near  far

(d)
Estimated depth map
for keyframe

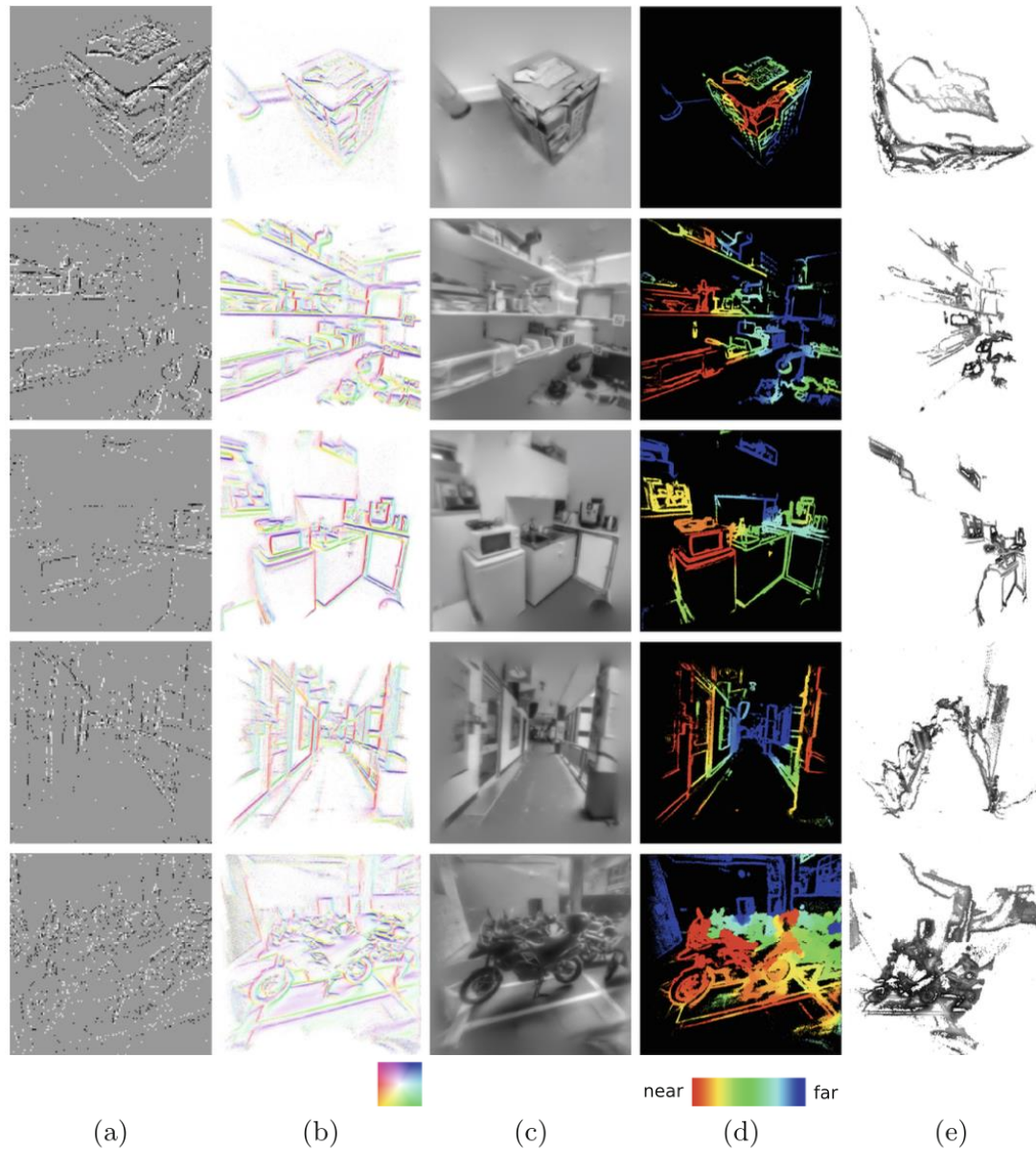


(e)
Semi-dense 3D point
cloud

Poorer performance in
occlusion boundaries

Poorer performance where
there are not enough edges or
textures that cause events

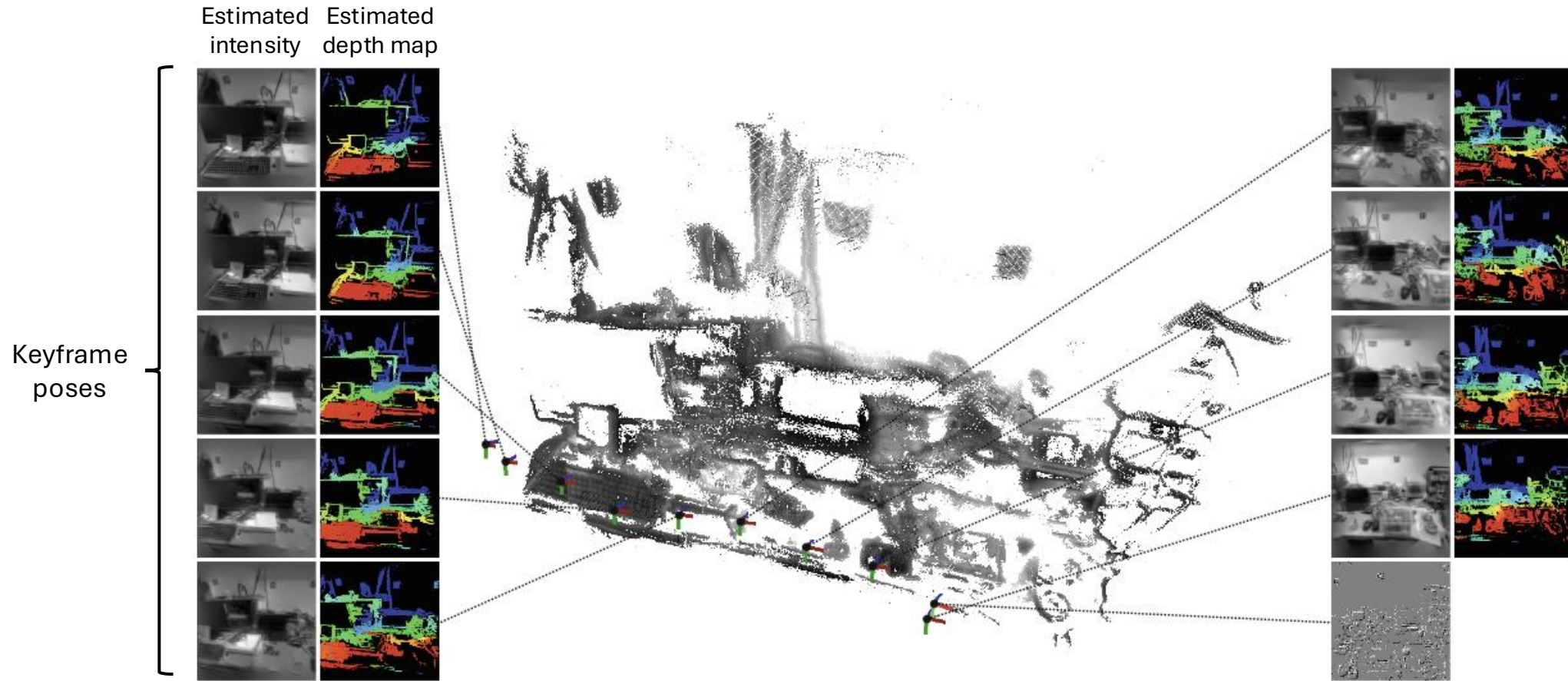
More single keyframe experiments



Multiple keyframes

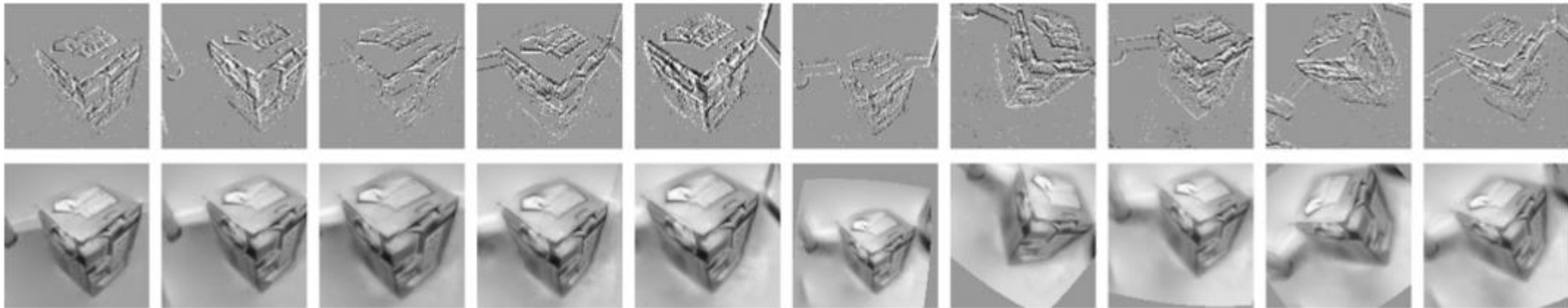
- To handle larger camera motions, the system creates multiple keyframes.
 - Once the camera moves too far from the current keyframe, it creates a new one by projecting all high confidence 3D points from the current keyframes into the new position

Mutliple keyframes



Video rendering

- Authors also claim that they can turn an event camera into a high speed and HDR artificial camera by rendering video frames based on ray-casting

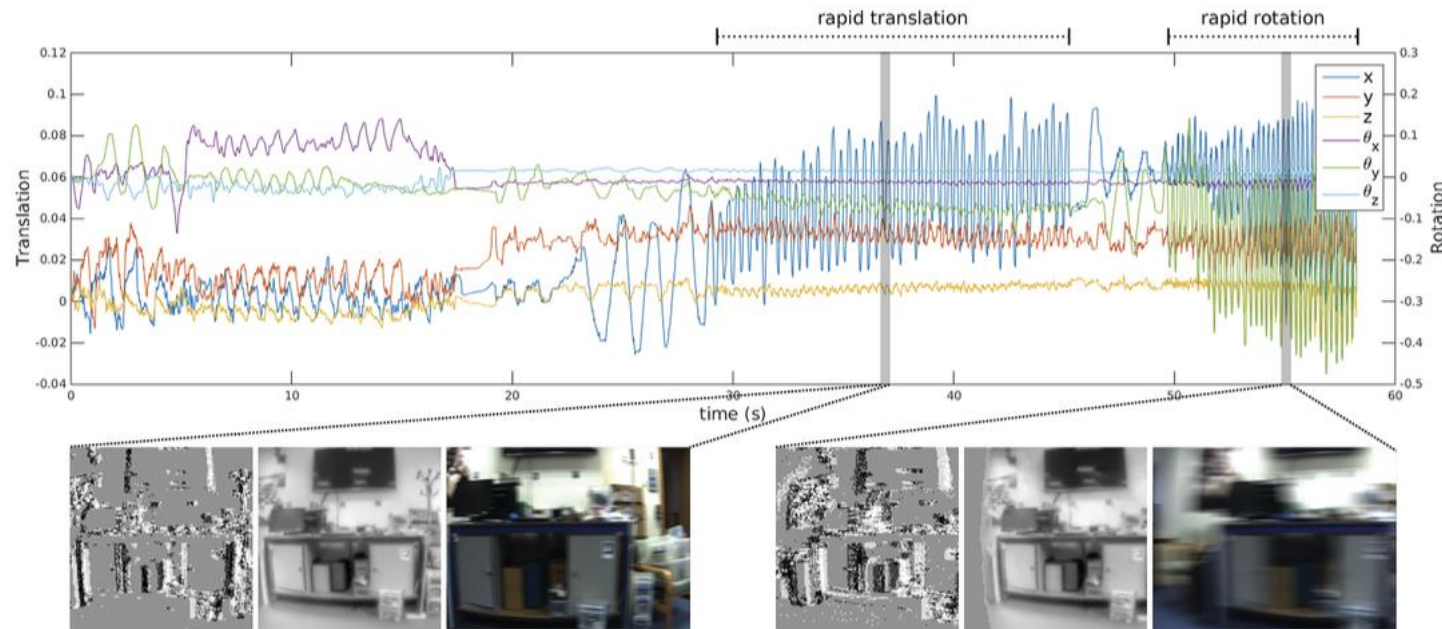


HDR video frames using an event camera

High speed tracking

- Authors tested the event camera with rapid motions

Estimated camera
pose history



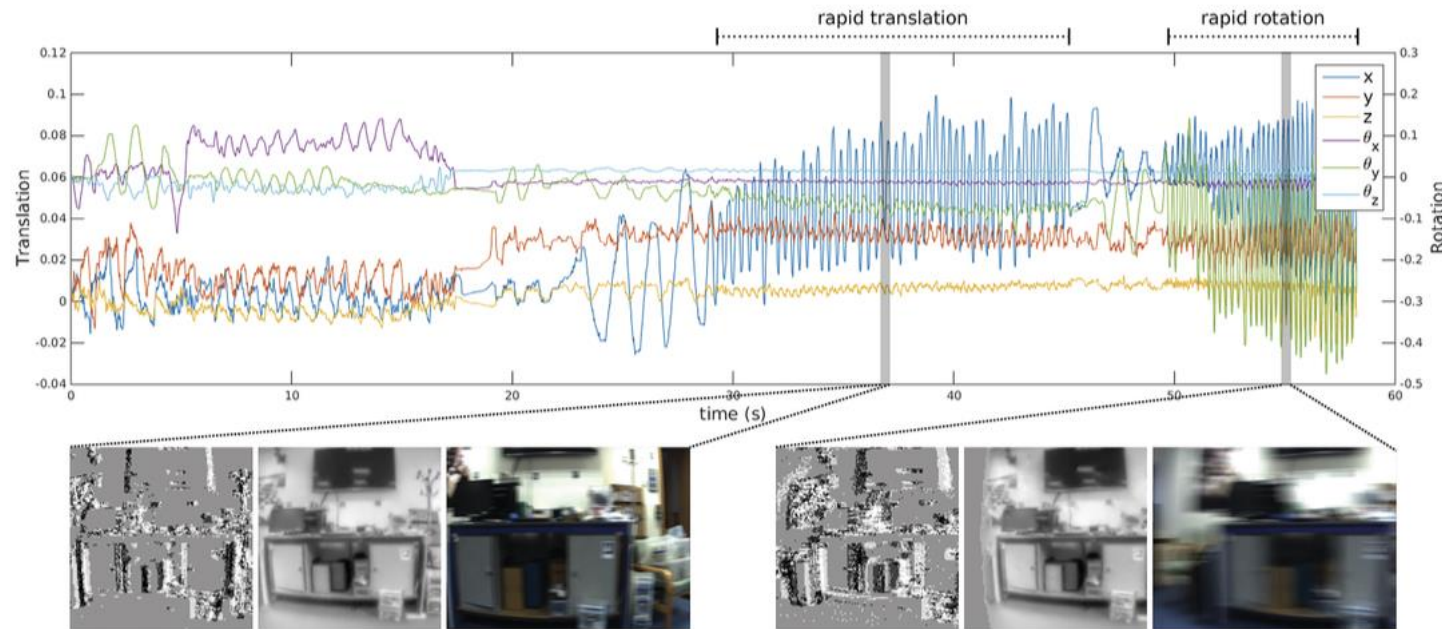
Event stream
visualization

Rendered
video frame
from event
camera

Standard
camera
video frame

High speed tracking

- Authors tested the event camera with rapid motions



- However, the paper's current implementation is not able to work with very high event rate (1M events per second) in real-time.

Discussion, Conclusion, and Key Takeaways

- Results are qualitative.
 - No ground truth measurements
 - No precise numerical metrics
- Assuming interleaved filters are correct is convenient approximation but not always valid
 - Inverse depth filter converges slowly
 - During that slow convergence period, other filters are working with poor depth information
- At the time of publication, however, this was the first system to conduct SLAM with no additional sensors, in real-time, and with no additional information on the scene (i.e., system worked with unstructured scenes)

Thank you!

